

基于Syslab的数值分析

目录

目录

1. 多项式计算

2. 数据插值与拟合

3. 数值积分与微分

4. 微分方程

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
侵权必究

1.1 多项式计算

Syslab中将多项式表示为行向量，其中包含按降幂排序的系数。

$$\boxed{p=[p_2,p_1,p_0]} \xrightarrow{\text{表示}} \boxed{p(x) = p_2x^2 + p_1x + p_0}$$

```
#创建一个向量以表示二次多项式 $p(x) = x^2 - 4x + 4$ 
julia> p = [1, -4, 4]
3-element Vector{Int64}:
 1
-4
 4

#创建一个向量以表示二次多项式 $p(x) = 4x^5 - 3x^2 + 2x + 33$ 
julia> p = [4, 0, 0, -3, 2, 33]
6-element Vector{Int64}:
 4
 0
 0
-3
 2
33
```

注意：

1. 多项式系数向量顺序按照次数由高到低
2. 多项式系数向量包含0次项系数，所以其长度为多项式最高次数加1
3. 必须将系数为0的多项式中间项输入到该向量中，因为0用作x的特定幂的占位符

1.2 多项式的四则运算

1. 多项式加减运算：系数向量相加减
2. 多项式乘法：使用`conv(p1,p2)`函数实现多项式相乘，`p1,p2`是两个多项式系数向量
3. 多项式除法：使用`q,r=deconv(p1,p2)`函数实现多项式相除，`q`为多项式`p1`除以`p2`的商式，`r`返回`p1`除以`p2`的余式，`q`和`r`仍是多项式系数向量。

```
#创建一个向量以表示二次多项式 $p_1(x) = x^2 - 4x + 4$ 
julia> p1 = [1,-4,4]
3-element Vector{Int64}:
 1
-4
 4

#创建一个向量以表示二次多项式 $p_2(x) = 3x^3 + 2x + 4$ 
julia> p2 = [3,0,2,4]
4-element Vector{Int64}:
 3
 0
 2
 4

#求解 $p_1(x)+p_2(x)$ 
julia> [0;p1]+p2
4-element Vector{Int64}:
 3
 1
-2
 8
```

```
#求解 $p_1(x)-p_2(x)$ 
julia> [0;p1]-p2
4-element Vector{Int64}:
-3
 1
-6
 0

#求解 $p_1(x)*p_2(x)$ 
julia> conv(p1,p2)
6-element Vector{Int64}:
 3
-12
14
-4
-8
16

#求解 $p_1(x)/p_2(x)$ 
julia> q,r=deconv(p1,p2)
([0.0], [1, -4, 4])
```

1.3 多项式的求导

Syslab中多项式求导使用polyder函数实现

1. 返回 p 中的系数表示的多项式的导数

`k=polyder(p)`



$$k(x) = \frac{d}{dx} p(x)$$

2. 返回多项式 a 和 b 的乘积的导数

`k=polyder(p,b)[3]`



$$k(x) = \frac{d}{dx} [a(x)b(x)]$$

3. 返回多项式 a 和 b 的商的导数

`q,d=polyder(a,b)`



$$k(x) = \frac{d}{dx} \left[\frac{a(x)}{b(x)} \right]$$

#创建一个向量以表示二次多项式 $p1(x) = x^2 - 4x + 4$

```
julia> p1 = [1, -4, 4]
```

```
3-element Vector{Int64}:
```

```
1
```

```
-4
```

```
4
```

#对多项式求导 $q1(x) = 2x - 4$

```
julia> q1=polyder(p)
```

```
2-element Vector{Int64}:
```

```
2
```

```
-4
```

#创建一个向量以表示二次多项式 $p2(x) = 2x + 4$, 求解

$q2(x) = \frac{d}{dx} [p1(x)p2(x)]$

```
julia> p2 = [2, 4]; q2=polyder(p1,p2)[3]
```

```
3-element Vector{Int64}:
```

```
6
```

```
-8
```

```
-8
```

#求解 $q, d = \frac{d}{dx} \left[\frac{p1(x)}{p2(x)} \right]$

```
#julia> q,d=polyder(p1,p2)
```

```
([0.5, 2.0, -6.0], [1.0, 4.0, 4.0])
```

©苏州同元软控信息技术有限公司版权所有，侵权必究

1.4 多项式的求值

1.polyval函数根据特定值计算多项式（标量）

```
#创建一个向量以表示二次多项式 $p(x) = x^2 - 4x + 4$ 
julia> p = [1,-4,4]
3-element Vector{Int64}:
 1
-4
 4

#计算p(2)的值
julia> polyval(p,2)
0
```

2.Polyvalm函数以矩阵方式计算多项式

```
#创建一个向量以表示二次多项式 $p(X) = X^2 - 4X + 4$ 
julia> p = [1,-4,4]
3-element Vector{Int64}:
 1
-4
 4

#创建方阵X
julia> X = [2 4 5; -1 0 3; 7 1 5]
3×3 Matrix{Int64}:
 2  4  5
-1  0  3
 7  1  5

#计算p(2)的值
julia> Y = polyvalm(p,X)
3×3 Matrix{Int64}:
31  -3  27
23   3  -2
20  29  47
```

@苏州同元软控信息技术有限公司版权所有，侵权必究

1.5 多项式的求根

Syslab中，使用roots函数对多项式求根。

roots(p):以列向量的形式返回 p 表示的多项式的根

```
#对方程  $3x^2-2x-4=0$  求解
```

```
julia> p = [3,-2,-4];r = roots(p)
```

```
2-element Vector{Float64}:
```

```
1.5351837584879962
```

```
-0.8685170918213299
```

```
#对方程  $x^4-1=0$  求解
```

```
julia> p = [1,0,0,0,-1];r = roots(p)
```

```
4-element Vector{ComplexF64}:
```

```
-1.0000000000000004 + 0.0im
```

```
8.326672684688674e-17 + 0.9999999999999996im
```

```
8.326672684688674e-17 - 0.9999999999999996im
```

```
0.9999999999999999 + 0.0im
```

©苏州同元软控信息技术有限公司
版权所有，侵权必究

未经许可，不得复印、传播或以其他方式使用

2. 数据插值与拟合

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

2.1 数据插值概念

插值最早来源于天体计算-由若干观测值（即节点）计算任意时刻星球的位置（即插值点和插值）的需要。

这就是说，我们只知道的一张观测数据表：

x_i	x_1	x_2	...	x_n
$f(x_i)$	$f(x_1)$	$f(x_2)$	...	$f(x_n)$

而不知道函数在其他点 x 上的取值，这时只能用一个经验函数 $y = g(x)$ 对真实函数 $y = f(x)$ 作近似

插值是在一组一直数据点的范围内添加新数据点的技术。可以使用插值来填充确实的数据、对现有数据进行平滑处理以及进行预测等。

2.1 数据插值使用方式

插值类型	函数名	简介
一维插值和 网格插值	interp1	一维数据插值（表查找）
	interp2	meshgrid 格式的二维网格数据插值
	interp3	meshgrid 格式的三维网格数据的插值
	interp _n	n 维插值
	griddedInterpolant	网格插值
	akima	Akima分段三次Hermite插值
	interpft	一维插值（FFT 方法）
	makima	修正 Akima 分段三次 Hermite 插值
	pchip	分段三次 Hermite 插值多项式 (PCHIP)
	spline	三次方样条数据插值
	interpolate	插值
	extrapolate	外插
	LagrangeInterp	拉格朗日插值
	NewtonInterp	牛顿插值
	ConstantInterpolation	最邻近插值
	LinearInterpolation	线性插值

插值类型	函数名	简介
一维插值和 网格插值	CubicSplineInterpolation	三次样条插值
	mkpp	生成分段多项式
	unmkpp	提取分段多项式详细信息
	ppval	计算分段多项式
	padecoeff	时滞的 Padé 逼近
网格创建	meshgrid2	创建二维网格
	meshgrid3	创建三维网格
	ndgrid	N 维空间中的矩形网格
散点插值	scatteredInterpolant	散点数据插值
	evaluate	计算插值方法在数据点处的值
	griddata	对二维或三维散点数据插值

PS:本课程详细介绍使用较多的一维数据插值 (*interp1*) 与 meshgrid 格式的二维网格数据插值(*interp2*), 其他函数使用详见帮助文档

2.2 一维插值

一维插值在曲线拟合和数据分析中具有重要的地位。在Syslab 中，一维插值由函数`interp1` 实现。该函数的调用格式为

$yi = \text{interp1}(x, y, xi, method)$

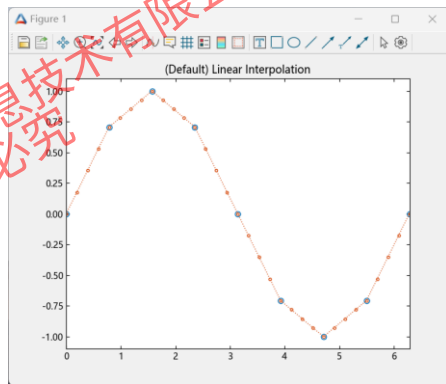
x y : 数据 x 坐标 y 坐标

xi : 待插值的位置

$method$: 采用的插值方法

该语句返回函数在点 xi 处的插值结果。

```
x = 0:pi/4:2*pi
v = sin.(x)
xq = 0:pi/16:2*pi
figure()
vq1 = interp1(x, v, xq)
plot(x, v, "o", xq, vq1, "-")
xlim([0, 2 * pi])
title("(Default) Linear Interpolation")
```



method常用方法有如下四种:

1.**linear**: 线性插值，默认方法。查询点处的插入值基于各维中邻近网格点处数值的线性插值

- 需要至少 2 个点
- 比最近邻点插值需要更多内存和计算时间

2.**nearest**: 最近邻点插值。查询点处的插入值是距样本网格点最近的值。

- 需要至少 2 个点
- 最低内存要求
- 最快计算时间

3.**pchip**: 分段3次埃尔米特插值。查询点处的插入值基于邻近网格点处数值的保形分段三次插值。

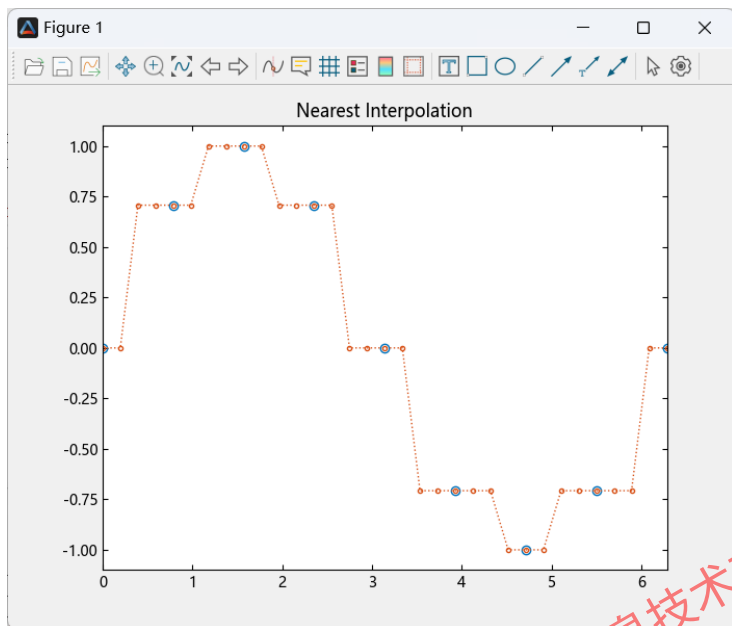
- 需要至少 4 个点
- 比 "linear" 需要更多内存和计算时间

4.**spline**: 根据数据点个数采取不同算法。

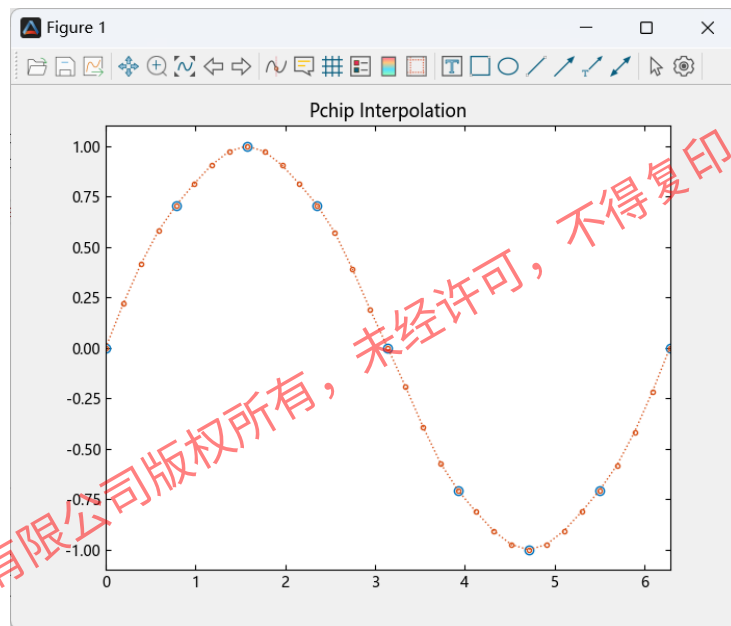
- 两个数据点: 线性插值
- 三个数据点: 抛物插值
- 四个数据点及以上: 使用非结终止条件的样条插值。查询点处的插入值基于各维中邻近网格点处数值的三次插值
- 需要至少 2 个点。
- 比 "pchip" 需要更多内存和计算时间

2.2 一维插值

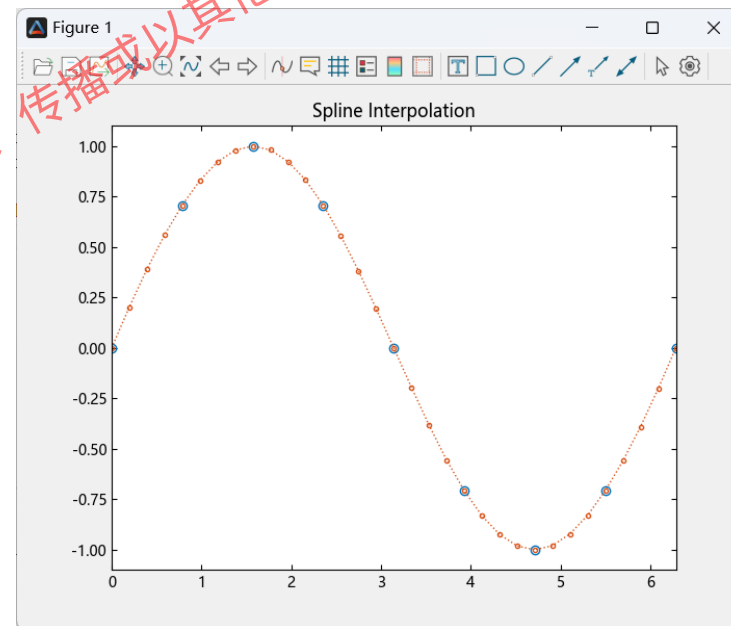
上页示例的其余三种插值方法，结果曲线如下：



Nearest插值



pchip插值



spline插值

@苏州同元软控信息技术有限公司
版权所有，侵权必究

传播或以其他形式使用

2.2 一维插值

interp1插值的局限性：首先，不能要求有独立变量范围以外的结果。

例如：interp1(hours, temps, 13.5)导致一个错误，因为hours在1到12之间变化，不报错误，但是计算的结果为NaN。因此需要介绍外插值。

外插值：当插值点落在已知数据集的外部，就需要对该点进行插值估算。

函数使用：通过interp1函数添加**extrapolation**参数，指明所用的插值算法也用于外插运算。也可以直接对数据集外的函数点赋值为**extrapolation**，一般赋值为NaN或者0。

```
julia> x=0:0.5:10;
```

```
julia> y=cos.(x);
```

```
julia> x1=10.5  
10.5
```

```
julia> y2=interp1(x,y,x1,"linear")  
-0.6809709019565264
```

```
julia> y2=interp1(x,y,x1,"linear",NaN)  
NaN
```

```
julia> y2=interp1(x,y,x1,"linear",1)  
1
```

2.2 二维插值与三维插值

有两个内置函数:interp2和interp3,用于分别处理二维和三维分段插值由它们的名字可知,这些函数的用法与 interp1类似

$yi = \text{interp2}(x, y, z, xi, yi, method)$

x y : 数据 x 坐标 y 坐标, 对应的是矩阵 z 中相应点的坐标

xi yi : 待插值的位置

$method$: 采用的插值方法

该语句返回函数在点 xi 处的插值结果。

注意, 可供选择的方法与interp1函数中的完全一样, 也就是linear、nearest、spline和cubic。和interp1函数一样, 如果省略参变量 $method$, 则默认为线性插值

2.3 三次样条插值

三次样条是实际中最常用的样条。而三次样条凭借其能给出满足光滑性要求的最简单表示，成为最受欢迎的样条。

线性样条的缺点：不光滑，两个样条的交点处（结点）斜率发生了剧烈不变化。

二次样条的缺点：最前面两个点是由直线连接的，最后一个区间中的样条摆动过大。

四次或更高次样条表现出高次多项式本身固有的不稳定性，所以我们一般不使用。

利用syslab内置函数spline 可以很容易地算出三次样条。它的一般用法为：

`s= spline(x,y,xq)`

返回与 xq 中的查询点对应的插值 s 的向量。s 的值由 x 和 y 的三次样条插值确定。

©苏州同元软控信息技术有限公司版权所有，侵权必究

2.2 插值-三次样条

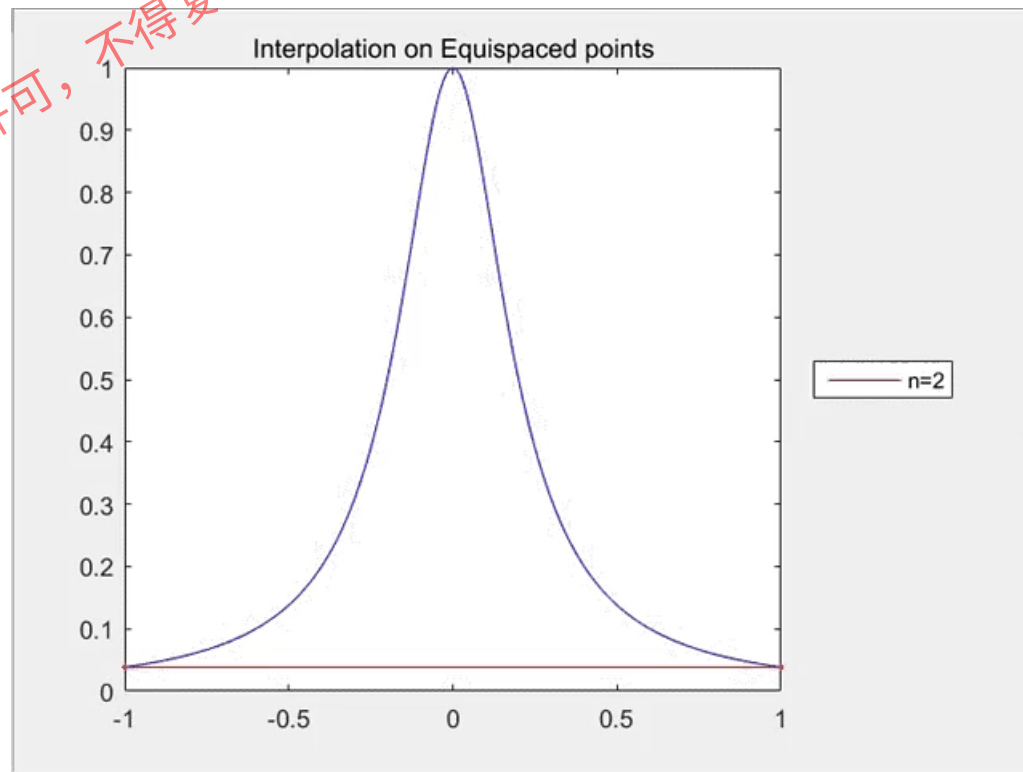
龙格现象

在科学计算领域，龙格现象（Runge）指的是对于某些函数，使用均匀节点构造高次多项式差值时，在插值区间的边缘的误差可能很大的现象。

它是由Runge在研究多项式差值的误差时发现的，表明：**并不是插值多项式的阶数越高，效果就会越好。**

最经典的一个例子是在均匀节点上对龙格函数

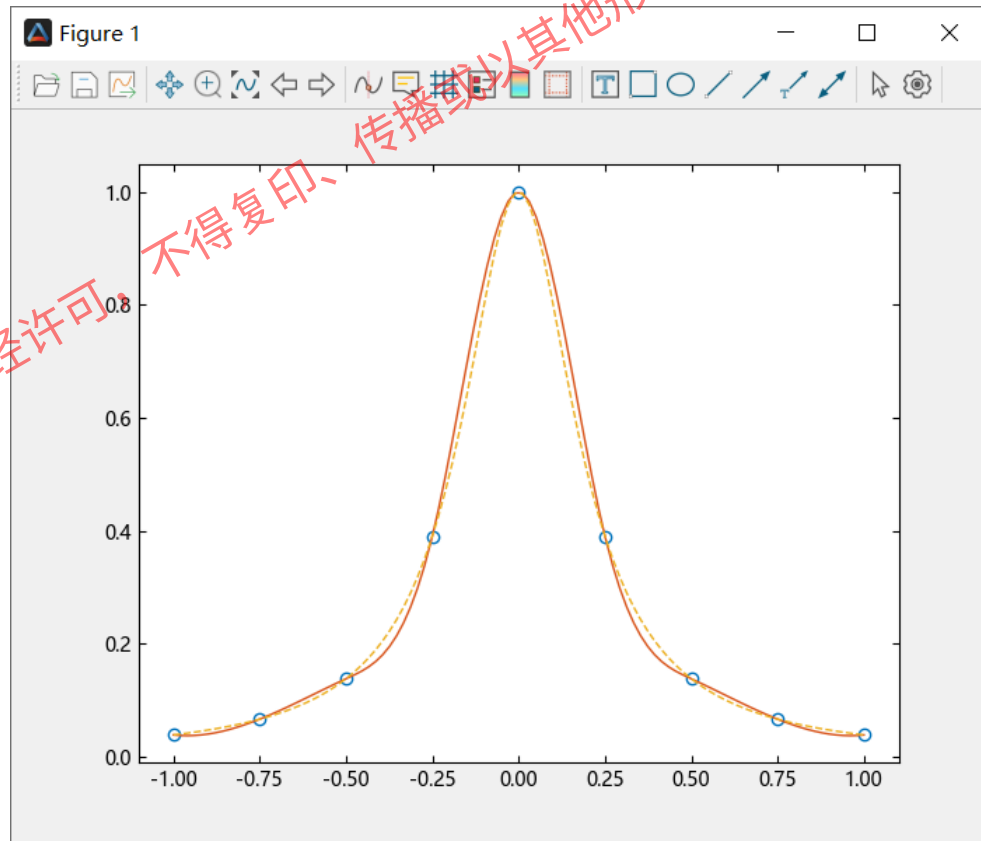
$$f(x) = \frac{1}{1 + 25x^2}, x \in [-1, 1]$$



2.3 三次样条插值

a.非节点样条

```
x=LinRange(-1,1,9)
y= @. 1/(1+25*x^2)
xx=LinRange(-1,1,100)
yy=spline(x,y,xx)
yr=@. 1/(1+25*xx^2)
plot(x,y,"o",xx,yy,xx,yr,"--")
```



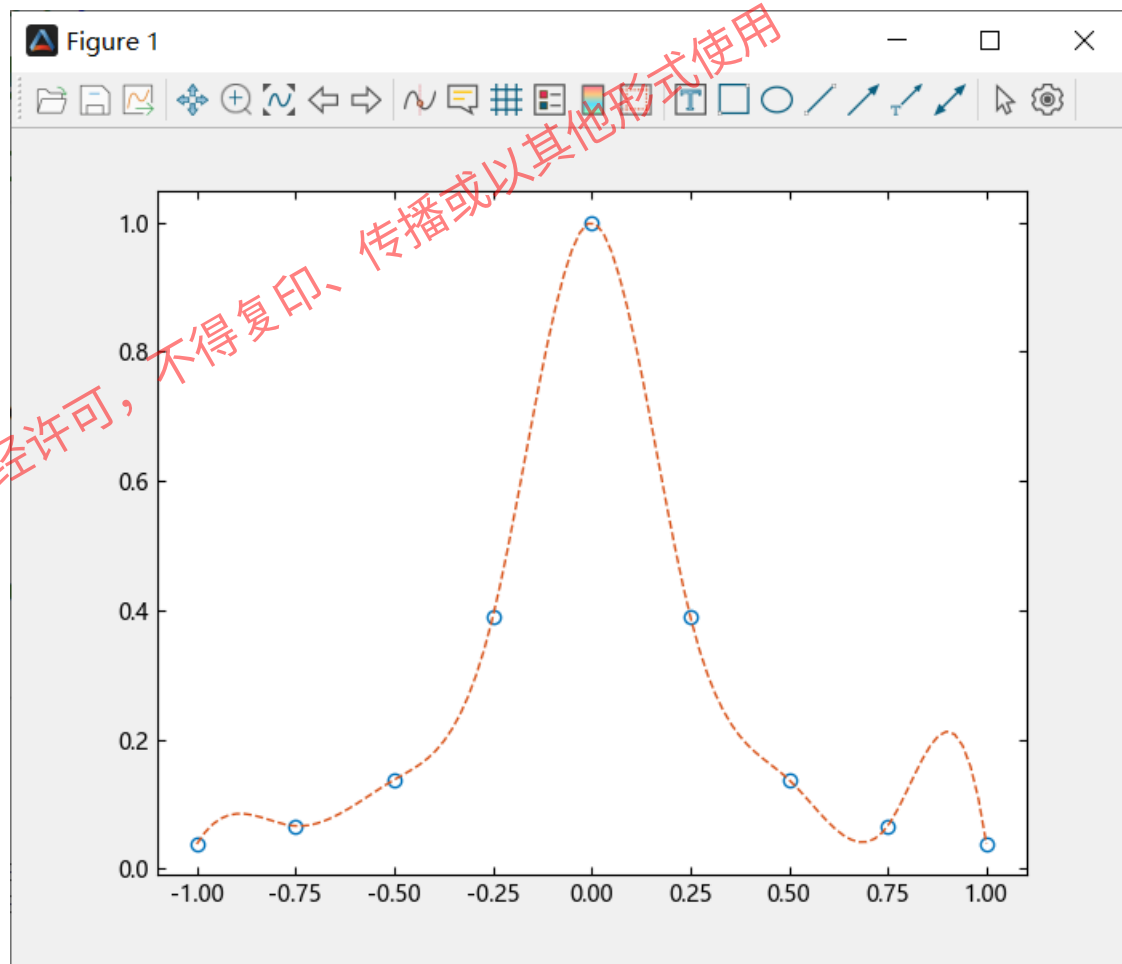
非结点样条可以很好地模拟龙格函数的图形，结点之间并没有产生大的震荡

2.3 三次样条插值

b. 固定样条

固定条件下，需要生成一个新向量 y_c ，它的第一个和最后一个元素为给定的阶导数值。然后用新向量进行样条拟合，并绘制图形：

```
x=LinRange(-1,1,9)
y= @. 1/(1+25*x^2)
y_new=copy(y)
xx = LinRange(-1, 1, 100)
yy = spline(x, y_new, xx)
yr = @. 1 / (1 + 25 * xx^2)
y = reshape(y,(1,9))
yyc=spline(x,[1 y -4])
plot(x,y_new,"o",xx,ppval(yyc, xx), "--")
```



由于边界处的斜率是我们人为指定的，所以这里的固定样条呈现出一些振荡。如果我们能预先知道真实的一阶导数值，那么固定样条的拟合效果则会变得很好。

2.4 数据插值应用示例

假设在一个曲面上以5厘米固定间隔收集温度数据，在每个方向上延伸20厘米。将数据分辨率提高十倍，使用interp2将温度数据插值到使用0.5厘米间隔的更精细的网格上。

#使用meshgrid2创建完整网格

```
X, Y = meshgrid2(0:5:20)
```

#创建一个矩阵来表示网格上的温度测量值，然后将数据绘制为曲面

```
T = [1 1 10 1 1  
     1 10 10 10 10  
     100 100 1000 100 100  
     10 10 10 10 1  
     1 1 10 1 1];
```

```
figure()
```

```
surf(X, Y, T)
```

```
plt_view(2)
```

#再次使用meshgrid创建由矩阵Xq和Yq表示的更精细的网格。然后，将interp2与原始网格、温度数据和新网格点结合使用，绘制生成的数据。

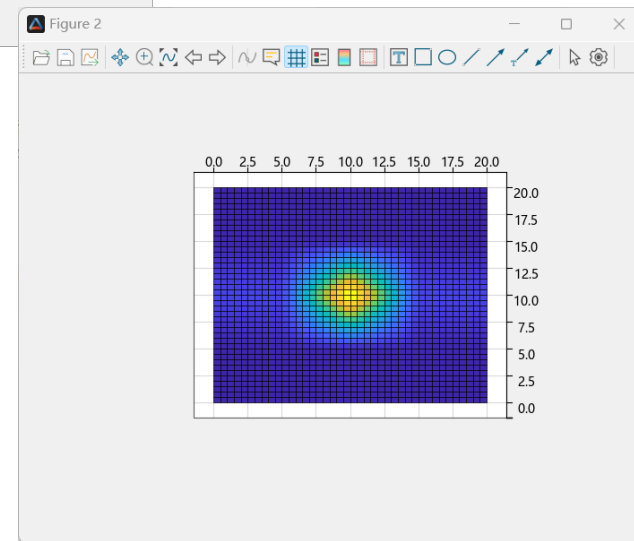
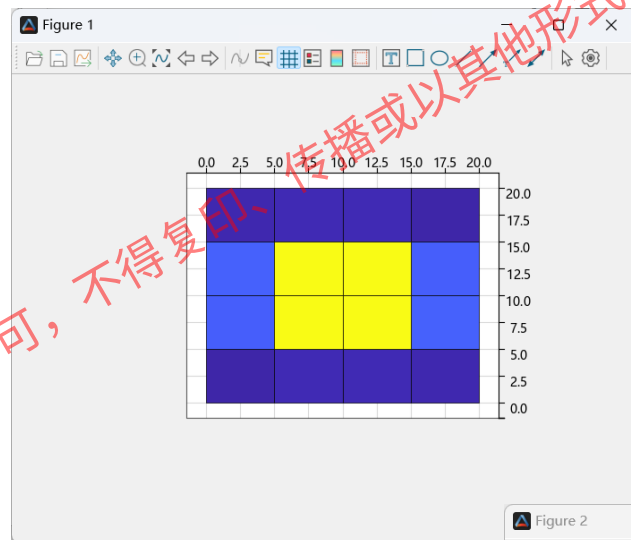
```
Xq, Yq = meshgrid2(0:0.5:20)
```

```
Tq = interp2(X, Y, T, Xq, Yq)
```

```
figure()
```

```
surf(Xq, Yq, Tq)
```

```
plt_view(2)
```



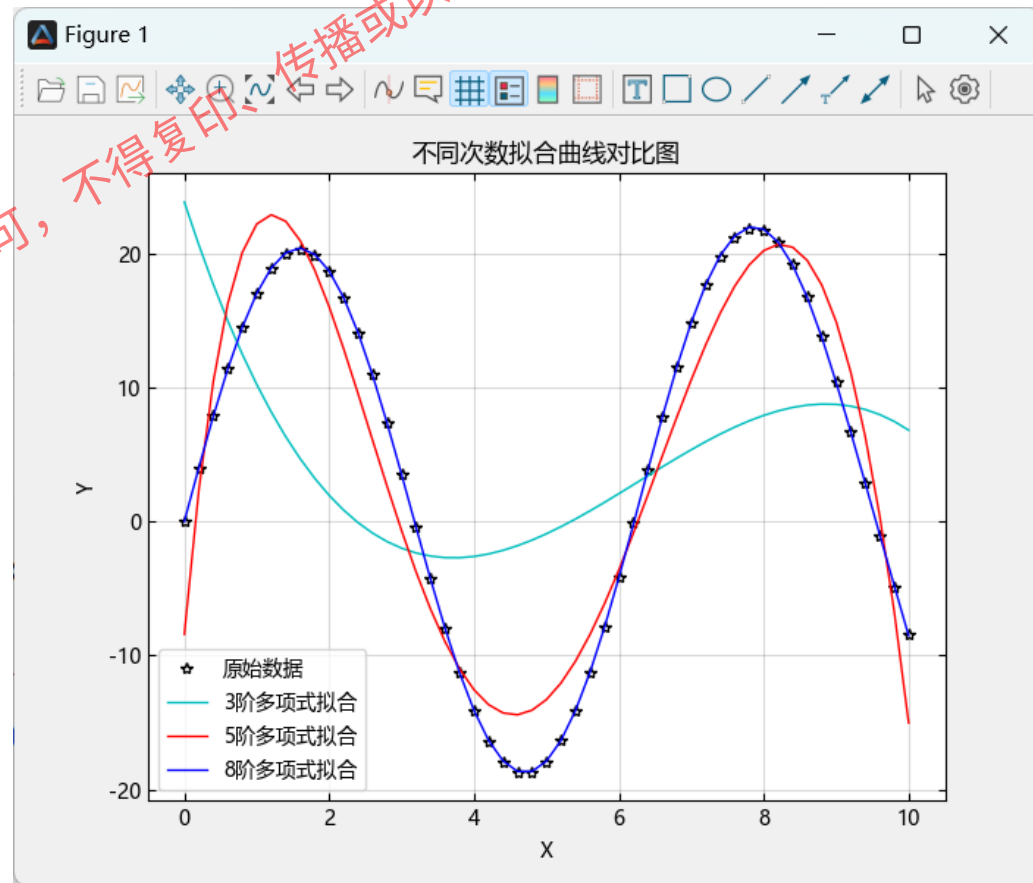
2.5 曲线拟合方法

曲线拟合是工程中经常用到的技术之一，Syslab中polyfit求出以最小二乘方式与一组数据拟合的多项式的系数。

- **p = polyfit(x,y,n)** : x和y是包含数据点的x和y坐标的向量，n是要拟合的多项式的次数

例：用3、5和8阶多项式拟合函数 $f(x) = 0.25x + 20\sin(x)$

```
x=0:0.2:10; y=0.25.*x+20.*sin.(x);
plot(x,y,"k*")
grid("on")
hold("on")
#3阶多项式拟合
p1, = polyfit(x, y, 3)
y1=polyval(p1,x)
#5阶多项式拟合
p2,= polyfit(x, y, 5)
y2=polyval(p2,x)
#8阶多项式拟合
p3, = polyfit(x, y, 8)
y3=polyval(p3,x)
#绘制曲线并设置曲线属性
plot(x,y1,"c-", x,y2, "r-", x,y3, "b-")
xlabel("X"); ylabel("Y")
legend("原始数据", "3阶多项式拟合", "5阶多项式拟合", "8阶多项式拟合", loc="best")
title("不同次数拟合曲线对比图")
```



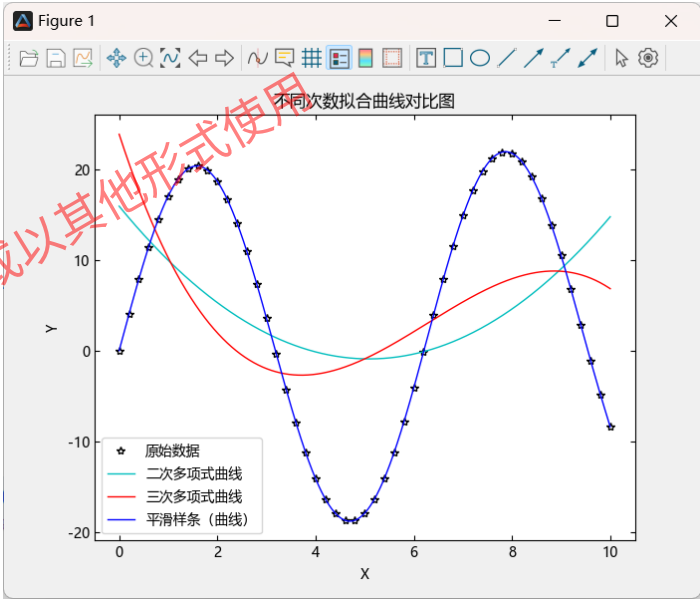
2.5 曲线拟合方法

Syslab中可以使用fit函数为数据拟合曲线或者曲面，使用方式如下：

- fit(fitType,x,y)**: 为 x,y 中的数据创建由 fitType 指定的模型的拟合

例：用3、5和8阶多项式拟合函数 $f(x) = 0.25x + 20\sin(x)$

```
x = 0:0.2:10;  
y = 0.25 .* x + 20 * sin.(x);  
plot(x, y, "k*")  
hold("on")  
#3阶多项式拟合  
fit1 = fit("poly2", x, y)  
plotfit(fit1, "c")  
#5阶多项式拟合  
fit2 = fit("poly3", x, y)  
plotfit(fit2, "r")  
#8阶多项式拟合  
fit3 = fit("smoothingspline", x, y)  
plotfit(fit3, "b")  
hold("off")  
#曲线属性  
xlabel("X")  
ylabel("Y")  
title("不同次数拟合曲线对比图")  
legend(["原始数据", "二次多项式曲线", "三次多项式曲线", "平滑样条 (曲线)"]); loc="best")
```



内置模型名称	描述
"poly1"	线性多项式曲线
"poly11"	线性多项式曲面
"poly2"	二次多项式曲线
"linearinterp"	分段线性插值
"cubicinterp"	分段三次插值
"smoothingspline"	平滑样条 (曲线)
"lowess"	局部线性回归 (曲面)

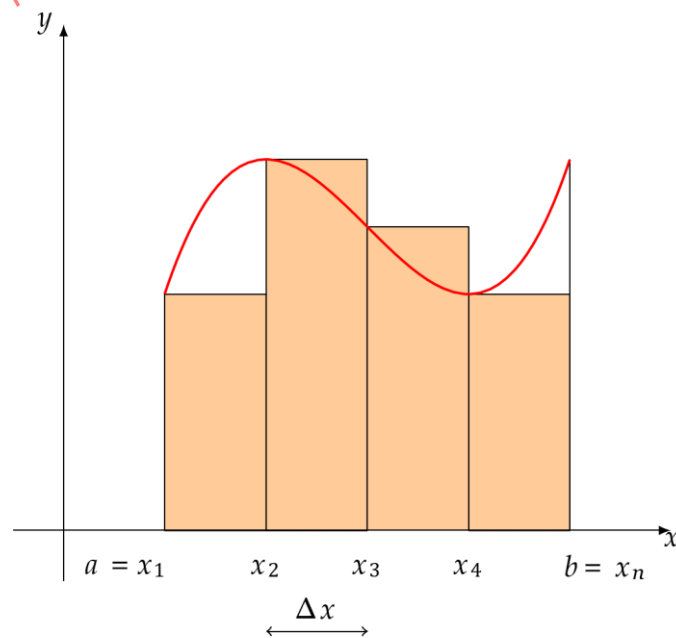
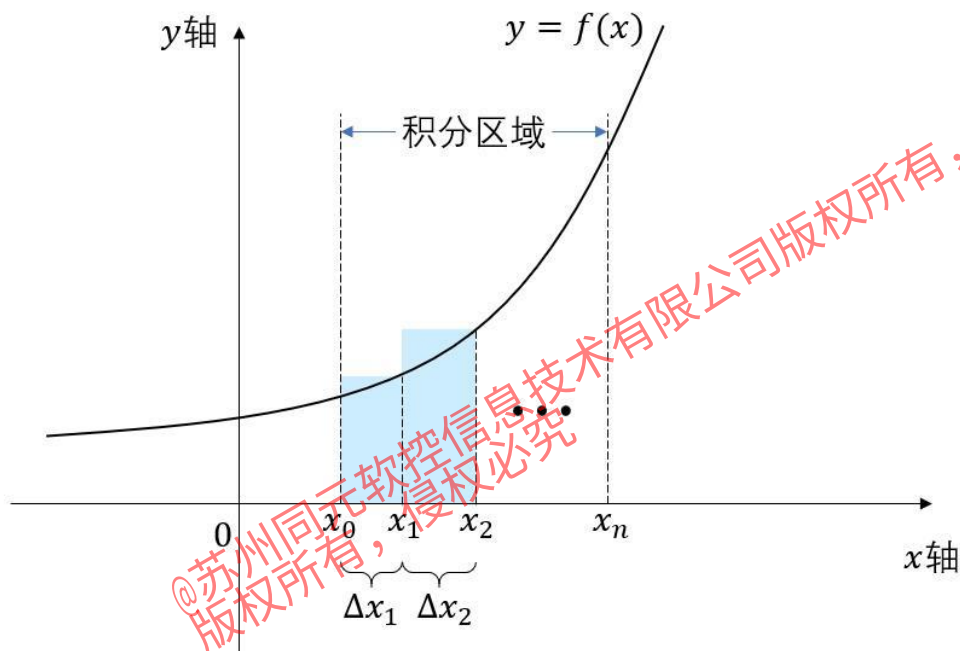
3. 数值积分与微分

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

3.1 数值积分

在数学分析中，许多定积分不能用已知的积分公式得到精确值，利用给定函数的定积分计算不总是可行的，数值积分是计算定积分数值的方法和理论，用于求定积分的近似值。

求解定积分的数值方法多种多样，如简单的梯形法、辛普生(Simpson)法、牛顿 - 柯特斯(Newton-Cotes)法等都是经常采用的方法。它们的基本思想都是将整个积分区间 $[a, b]$ 分成 n 个子区间 $[x_i, x_{i+1}]$, $i = 1, 2, \dots, n$, 其中 $x_1 = a$, $x_{n+1} = b$ 。这样求定积分问题就分解为求和问题。



3.1 数值积分-一重积分

Syslab中的一重积分函数: quad、quadl、quadgk、integral

`q, e = quad(fun, a, b)`

- simpson公式, 适用于精确度较低, 被积函数平滑性较差的数值积分

`Q, fcnt = quadl(fun, a, b, tol, trace)`

- Lobatto公式, 精确度要求较高, 被积函数也更为平滑的数值积分, fcnt为函数计算次数

`quadgk(f, a, b, c...; rtol=sqrt(eps), atol=0, maxevals=10^7, order=7, norm=norm)`

- Gauss-Kronrod数值积分, 函数的精确度最高, 会产生震荡被积函数。它支持无限区间并且可以处理端点处的适度奇异性。它还支持沿分段线性路径的围道积分

`q, e = integral(fun, xmin, xmax; Name=Value)`

- 使用全局自适应积分和默认误差容限在xmin至xmax间以数值形式为函数fun求积分。指定具有一个或多个Name, Value对组参数的其他选项。例如, 指定 rtol, 后跟实数, 可调整相对误差

3.1 数值积分-一重积分

例：求积分 $\int_0^{\pi/2} \sin(x) dx = 1$ ，分别用quad、quadl、quadgk和integral函数

#函数定义

```
Julia> fh(x)=sin(x)
```

#精度要求低，自适应simpson数值积分

```
Julia> I1=quad(fh,0,pi/2)
```

```
(0.9999999999999999, 1.1102230246251565e-16)
```

#精度要求高，自适应Lobatto数值积分

```
Julia> I2=quadl(fh,0,pi/2)
```

```
(0.9999999999917477, 18)
```

#精度要求低，自适应Lobatto数值积分

```
Julia> I3=quadl(fh,0,pi/2,1e-12)
```

```
(0.9999999999999999, 48)
```

#自使用Gauss-Kronrod数值积分，精度高

```
Julia> I4=quadgk(fh,0,pi/2)
```

```
(0.9999999999999999, 1.1102230246251565e-16)
```

```
Julia> I1=integral(fh,0,pi/2)
```

```
(0.9999999999999999, 1.1102230246251565e-16)
```

©苏州同元软件技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

3.1 数值积分-二重积分

Syslab中的二重积分函数: quad2d、dblquad、integral2

$q, E = \text{quad2d}(\text{fun}, a, b, c, d; \text{Key}=\text{Value})$

- 逼近 $\text{fun}(x, y)$ 在平面区域 $a \leq x \leq b$ 和 $c(x) \leq y \leq d(x)$ 上的积分。边界 c 和 d 均可为标量或函数句柄并返回绝对误差 $E = |q - I|$ 的逼近上限, 其中 I 是积分的确切值。指定具有一个或多个 $\text{Key}=\text{Value}$ 关键字参数的其他选项。例如, 您可以指定 AbsTol 和 RelTol 来调整算法必须满足的误差阈值

$q = \text{dblquad}(\text{fun}, x_{\min}, x_{\max}, y_{\min}, y_{\max}, \text{tol}, \text{method})$

- 调用 quad 函数来计算 $x_{\min} \leq x \leq x_{\max}$, $y_{\min} \leq y \leq y_{\max}$ 矩形区域上的二重积分 $\text{fun}(x, y)$, 输入参数 fun 是一个函数句柄, 它接受标量 x , 标量 y , 并返回被积函数值, 使用容差 tol 代替默认值 $1.0e-6$, 使用指定为 method 的求积法函数代替默认值 quad 。 method 的有效值为 quad1 或用户指定的求积法的函数句柄, 该句柄与 quad 和 quad1 具有相同的调用顺序

$q, e = \text{integral2}(\text{fun}, x_{\min}, x_{\max}, y_{\min}, y_{\max}; \text{Name}=\text{Value})$

- 在平面区域 $x_{\min} \leq x \leq x_{\max}$ 和 $y_{\min}(x) \leq y \leq y_{\max}(x)$ 上逼近函数 $z = \text{fun}(x, y)$ 的积分。指定具有一个或多个 $\text{Name}, \text{Value}$ 对组参数的其他选项

3.1 数值积分-二重积分

例：求二重定积分 $I = \int_{-1}^1 \int_{-2}^2 e^{-x^2/2} \sin(x^2 + y) dx dy$

#函数定义

```
Julia> fh(x,y)=exp(-x.^2/2).*sin(x.^2+y)
```

```
Julia> I1=quad2d(fh,-2,2,-1,1)
```

```
(1.5744981414682058, 1.0865901790802828e-6)
```

```
Julia> I2= dblquad(fh, -2,2,-1,1, 1e-10,quadl)
```

```
1.5744931897449437
```

```
Julia> I3=integral2(fh,-2,2,-1,1, AbsTol=1e-8,RelTol=1e-12)
```

```
(0.8485131285302074, 0.000566300741526958)
```

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

版权所有，侵权必究

3.1 数值积分-三重积分

Syslab中的三重积分函数: quad3d、triplequad、integral3

`q,e = quad3d(fun,xmin,xmax,ymin,ymax,zmin,zmax,Name=Value)`

- 在区域 $x_{\min} \leq x \leq x_{\max}$ 、 $y_{\min}(x) \leq y \leq y_{\max}(x)$ 和 $z_{\min}(x,y) \leq z \leq z_{\max}(x,y)$ 逼近函数 $z = \text{fun}(x,y,z)$ 的积分, 指定具有一个或多个 Name, Value 对组参数的其他选项。

`q = triplequad(fun, xmin, xmax, ymin, ymax, zmin, zmax, tol, method)`

- 对三维矩形区域 $x_{\min} \leq x \leq x_{\max}$, $y_{\min} \leq y \leq y_{\max}$, $z_{\min} \leq z \leq z_{\max}$ 求三重积分 $\text{fun}(x,y,z)$, 输入参数 fun 是一个函数句柄, 它接受标量 x, 标量 y, 标量 z 并返回被积函数值, 使用容差 tol 代替默认值 $1.0e-6$, 使用指定为 method 的求积法函数代替默认值 quad。

`q,e=integral3(fun,xmin,xmax,ymin,ymax,zmin,zmax,Name=Value)`

- 在区域 $x_{\min} \leq x \leq x_{\max}$ 、 $y_{\min}(x) \leq y \leq y_{\max}(x)$ 和 $z_{\min}(x,y) \leq z \leq z_{\max}(x,y)$ 逼近函数 $z = \text{fun}(x,y,z)$ 的积分, 指定具有一个或多个 Name, Value 对组参数的其他选项。

3.1 数值积分-三重积分

例：用数值方程求解三重积分 $I = \int_0^1 \int_0^\pi \int_0^\pi 4xze^{-x^2y-z^2} dz dy dx$

#函数定义

```
Julia> fh(x,y,z)=4*x.*z.*exp(-(x.^2).*y-z.^2)
```

#使用quad3d函数对三重积分进行数值计算

```
Julia> q,e = quad3d(fh,0,1,0,pi, 0,pi,atol=1e-12)  
(2.5356917741148663, 9.999969847071944e-13)
```

#使用triplequad函数对三重积分进行数值计算，并设置积分法为quadl

```
Julia> q = triplequad(fh,0,1,0,pi, 0,pi,1e-10, quadl)  
1.7327622230309039
```

#使用integral3函数对三重积分进行数值计算

```
Julia> q,e=integral3(fh,0,1,0,pi,0,pi,atol=1e-8,rtol=1e-12)  
(2.535691774124124, 9.99789315994994e-9)
```

#指定积分法

```
Julia> q,e=integral3(fh,0,1,0,pi,0,pi,atol=1e-8,rtol=1e-  
12,method=:cuhre)  
(2.5356917791362537, 9.25601013761248e-9)
```

3.1 数值积分-向量化积分

Syslab中的向量化积分函数: `quadv`、`integral`

`Q,fcnt = quadv(fun,a,b,tol,trace)`

- 使用递归自适应Simpson积分法求取复数数组值函数`fun`从`a`到`b`的近似积分，误差小于 $1.e-6$ 并返回函数计算数。`fun`是函数句柄。函数`Y=fun(x)`应接受标量参数`x`并返回数组结果`Y`，其分量是在`x`处计算的被积函数。范围`a`和`b`必须是有限的，对所有积分使用绝对误差容限`tol`，代替默认值 $1.e-6$ ，`trace`指定为`true`时在递归期间显示`[fcnt a b-a Q[1]]`的值。

`q,e = integral.(fun, xmin,xmax;Name = Value)`

- 使用全局自适应积分和默认误差容限在`xmin`至`xmax`间以数值形式为函数`fun`求积分。指定具有一个或多个`Name,Value`对组参数的其他选项。例如，指定 `rto1`，后跟实数，可调整相对误差。

例：求函数 $f(x) = \frac{1}{n+x}$ 在`n`取1~10情况下，`x`从0~1的积分

#函数定义

```
Julia> fh= (x,n) -> 1 ./((1:n).+x)
```

```
Julia> Q,fcnt = quadv(x-> fh(x,10),0,1)
```

```
([0.69314719986297, 0.4054651083775654, 0.28768207247245975, 0.22314355131746128, 0.1823215567947161,  
0.15415067982748842, 0.1335313926246057, 0.11778303565641765, 0.10536051565784188, 0.09531017980433254], 17)
```

```
Julia> q,e= integral.(x-> fh(x,10),0,1)
```

```
([0.6931471805599453, 0.4054651081081644, 0.2876820724517809, 0.22314355131420976, 0.18232155679395462,  
0.1541506798272583, 0.13353139262452263, 0.11778303565638346, 0.10536051565782631, 0.09531017980432487],  
1.99318352275991e-11)
```

3.1 数值积分-离散数据积分

$Q = \text{trapz}(X,Y,\text{dim})$: 梯形数值积分, 通过已知参数 X,Y 按 dim 维使用梯形公式进行积分。

例1:

```
Julia> x=0:pi/100:pi/2;  
Julia> y=sin.(x);  
Julia> Q = trapz(x,y)  
0.968509576339474
```

例2:

#积分区间定义

```
Julia> x=-3:0.01:3;
```

```
Julia> y=-5:0.01:5;
```

#生成网格

```
Julia> X,Y=meshgrid2(x,y);
```

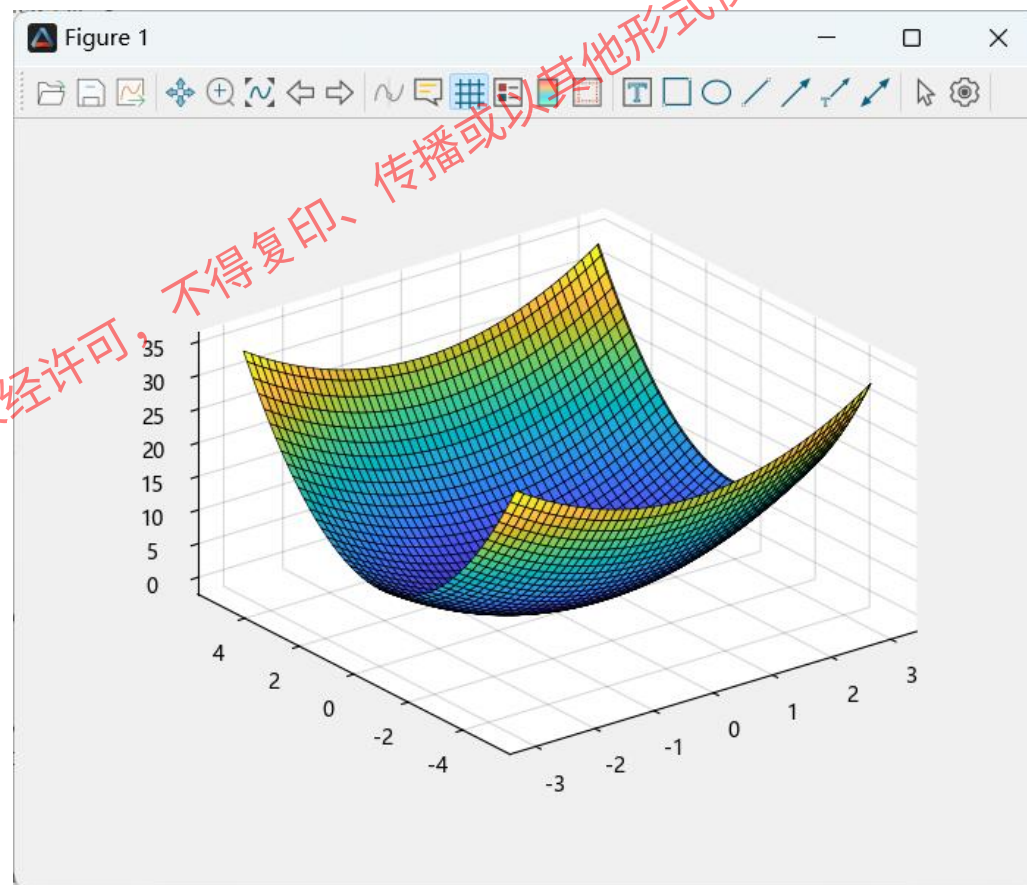
```
Julia> F=X.^2+Y.^2;
```

```
Julia> surf(X,Y,F)
```

#使用trapz先对x求积分以生成列向量。然后, y上的积分可将列向量减少为单个标量

```
Julia> Q=trapz(y,trapz(x,F,2))  
1×1 adjoint(::Vector{Float64}) with eltype Float64:  
680.002
```

#trapz 稍微高估计确切答案680, 因为 $f(x,y)$ 是向上凹的



3.2 数值微分

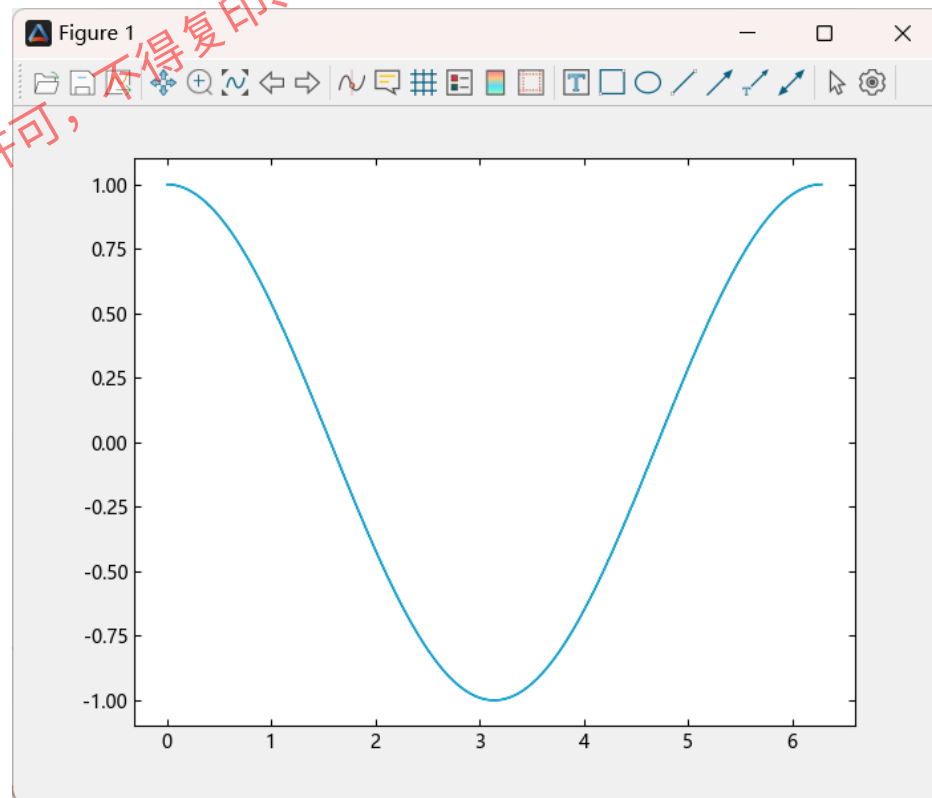
函数 $f(x)$ 在点 x_0 的微分接近于函数在该点的差分，而 f 在点 x 的导数接近于函数在该点的差商

Syslab中提供了求向前差分的函数diff，调用格式如下：

- $D=\text{diff}(x)$:计算向量 x 的向前差分， $dx(i)=x(i+1)-x(i), i=1,2,\dots,n-1$ 。
- $D=\text{diff}(x,n)$:计算向量 x 的 n 阶向前差分。例如， $\text{diff}(x,2)=\text{diff}(\text{diff}(x))$ 。
- $D=\text{diff}(x,n,\text{dim})$:计算矩阵 A 的 n 阶差分， $\text{dim}=1$ 时（默认状态），按行计算差分； $\text{dim}=2$ 时，按列来计算差分

例：设 $f(x) = \sin(x)$ ，在 $[0,2\pi]$ 范围内随机采样，计算 $f'(x)$ 的近似值，并与理论值 $f'(x) = \cos(x)$ 进行比较

```
#随机采样
x=[0;sort(2*pi*rand(5000));2pi]
y=sin.(x)
#求f在x处的导数值
f1=diff(y)./diff(x)
f2=cos.(x[1:end-1])
#两个解的曲线绘制
plot(x[1:end-1],f1,x[1:end-1],f2)
#近似值解与理论值的比较
d=norm(f1-f2)
#ans=0.044821948319216105
```



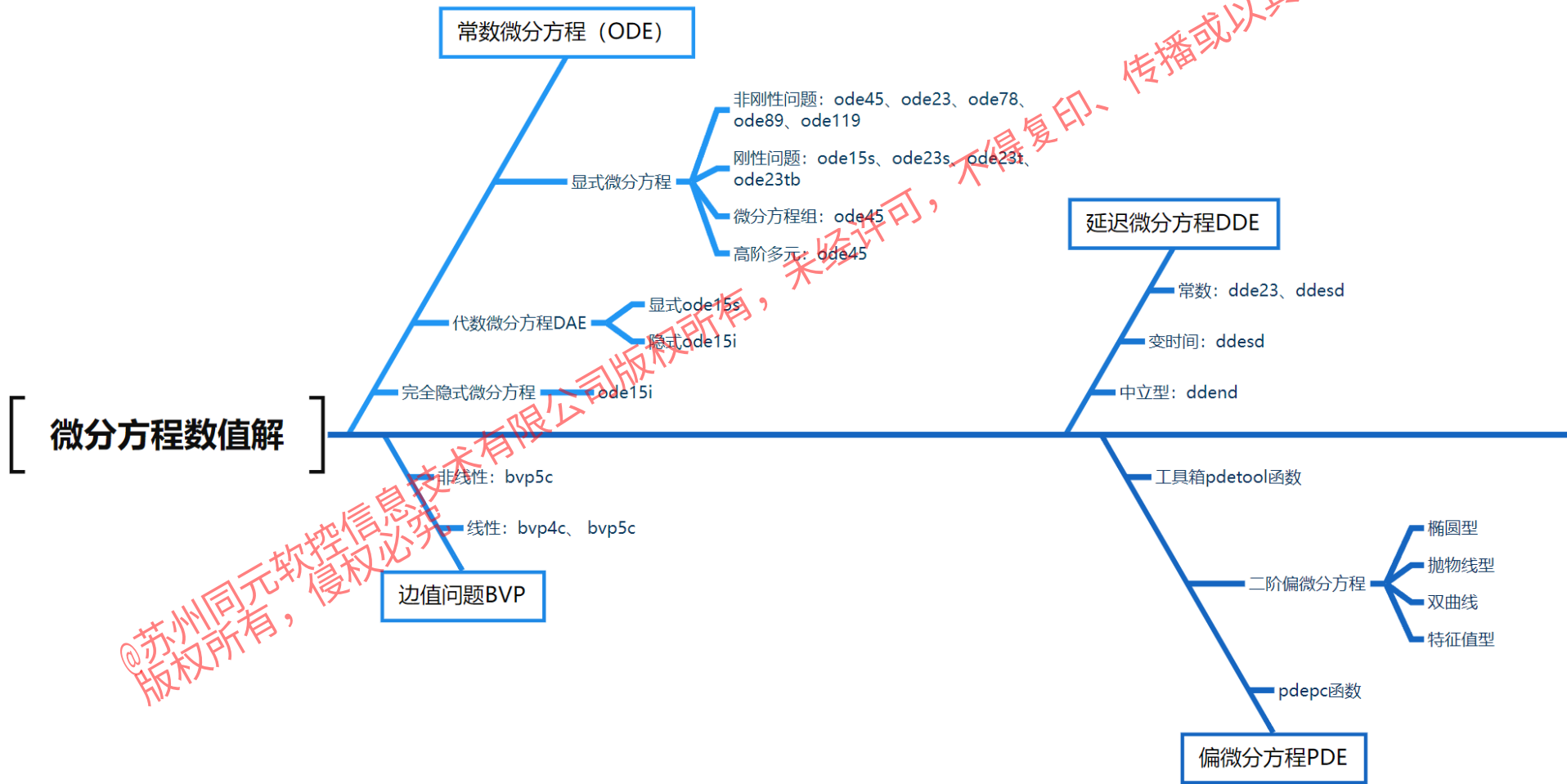
4. 微分方程

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

4.1 微分方程

微分方程是指含有未知函数及其导数的关系式。

微分方程板块函数提供ODE(常微分方程)、DDE(时滞微分方程)、BVP(边界值问题)、PDE（偏微分方程）等微分方程求解的解决方案



4.2.1 常微分方程的数值求解

□ 常微分方程求解的一般概念

常微分方程

常微分方程 (ODE) 包含与一个自变量 t (通常称为时间) 相关的因变量 y 的一个或多个导数。此处用于表示 y 关于 t 的导数的表示法对于一阶导数为 y' , 对于二阶导数为 y'' , 依此类推。ODE 的阶数等于 y 在方程中出现的最高阶导数。

例如, 这是一个二阶 ODE:

$$y'' = 9y$$

求解常微分方程初值问题就是寻找函数 $y(t)$ 使之满足如下方程:

$$y' = f(t, y), t_0 < t < b$$

$$y(t_0) = y_0$$

所谓其数值解法, 就是求 $y(t)$ 在离散结点 t_n 处的函数近似值 y_n 的方法, $y_n \sim y(t_n)$

这些近似值称为常微分方程初值问题的数值解。相邻两个结点之间的距离称为步长。

4.2.1 常微分方程的数值求解

□ 常微分方程求解的一般概念

- 单步法: 在计算 y_{n+1} 时只用到前一步的 y_n , 因此在有了初值之后, 就可以逐步往下计算。其代表是龙格-库塔 (Runge-Kutta) 法。
- 多步法: 在计算 y_{n+1} 时,除了用到前一步的值 y_n 之外,还要用到 y_{n-p} ($p = 1, 2, \dots, k, k > 0$) 的值, 即前面的 k 步。其代表就是亚当斯(Adams)法。

在每一步, 求解器都对之前各步的结果应用一个特定算法。在第一个这样的时间步, 初始条件将提供继续积分所需的必要信息。

最终结果是, ODE 求解器返回一个时间步向量 $t = [t_0, t_1, t_2, \dots, t_f]$ 以及在每一步对应解 $y = [y_0, y_1, y_2, \dots, y_f]$ 。

4.2.1 常微分方程的数值求解

□ 常微分方程求解的一般概念

Syslab提供了多个求常微分方程初值问题数值解的函数，一般调用格式为：

$$[t, y] = \text{solver}(\text{filename}, \text{tspan}, y0, \text{option})$$

其中， t 和 y 分别给出时间向量和相应的数值解。

solver 为求常微分方程数值解的函数。

filename 是定义 $f(t, y)$ 的函数名，该函数必须返回一个列向量。

tspan 形式为 $[t0, tf]$ ，表示求解区间。

$y0$ 是初始状态向量。

Option 是可选参数用于设置求解属性，常用的属性包括：相对误差值 RelTol (默认值是 10^{-3})和绝对误差值 AbsTol (默认值是 10^{-6})。

4.2.1 常微分方程的数值求解

□ 常微分方程求解的一般概念

常微分方程数值求解函数的统一命名格式：

odenxxx

- ode是Ordinary Differential Equation的缩写，是常微分方程
- nn是数字，代表所用方法的阶数。

例如：ode23采用2阶龙格-库塔(Runge-Kutta) 算法，用3阶公式做误差估计来调节步长，具有低等精度ode45采用4阶龙格-库塔算法，用5阶公式做误差估计来调节步长，具有中等精度。

- xx是字母，用于标注方法的专门特征。

例如：ode15s、ode23s中的‘s’代表 (Stiff)，表示函数适用于刚性方程。

4.2.1 常微分方程的数值求解

常微分方程求解的一般概念

非刚性求解器

函数名	采用方法	简介
ode45	4阶或者5阶的Runge-Kutta算法，中精度	求解非刚性微分方程 - 中阶方法
ode23	2阶或者3阶的Runge-Kutta算法，低精度	求解非刚性微分方程 - 低阶方法
ode78	7阶或者8阶的Runge-Kutta算法，高精度	求解非刚性微分方程 - 高阶方法
ode89	8阶或者9阶的Runge-Kutta算法，高精度	求解非刚性微分方程 - 高阶方法
ode113	Adams算法，精度可到 $10^{-3} \sim 10^{-6}$ ，时间比ode45短	求解非刚性微分方程 - 变阶方法

刚性求解器

函数名	采用方法	简介
ode15s	Gear 's反向数值微分算法，中精度	求解刚性微分方程和 DAE - 变阶方法
ode23s	2阶Rosebrock算法，低精度，时间比ode15s短	求解刚性微分方程 - 低阶方法
ode23t	梯形算法	求解中等刚性的 ODE 和 DAE - 梯形法则
ode23tb	梯形算法，低精度，时间比ode15s短	求解刚性微分方程 - 梯形法则 + 后向差分公式

完全隐式求解器

函数名	简介
ode15i	解算全隐式微分方程 - 变阶方法
decic	为 ode15i 计算一致的初始条件

4.2.2 常微分方程的数值求解-显式微分方程

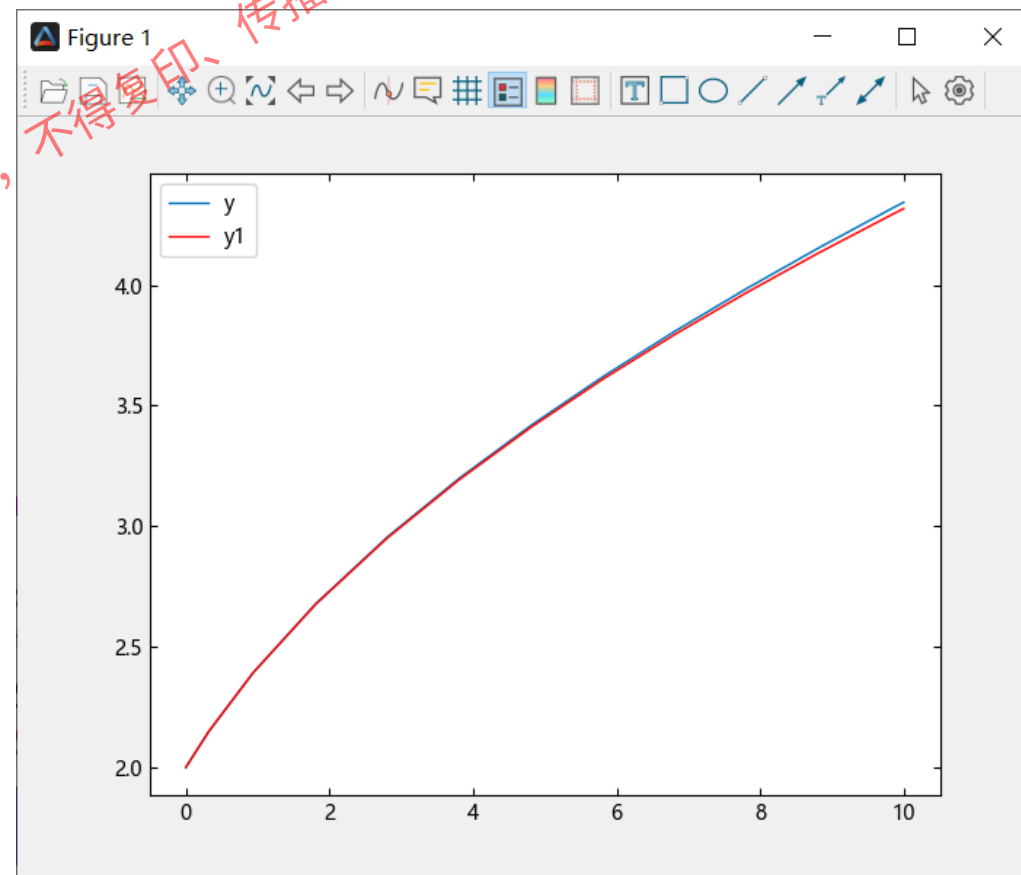
显式微分方程-非刚性问题

➤ 示例1:

求解微分方程的初值问题，并与精确解 $y(t) = \sqrt{t+1} + 1$ 比较

$$y' = \frac{y^2 - t - 2}{4(t+1)}, \quad 0 \leq t \leq 10$$
$$y(0) = 2$$

```
function model(t, y)
    dydt = (y.^2 - t - 2) ./ (4 .* (t + 1))
    return dydt
end
t, y, = ode23(model, [0 10], 2)
y1 = sqrt.(t.+1) .+ 1
plot(t, y, "b", t, y1, "r")
legend("y", "y1")
```



4.2.2 常微分方程的数值求解-显式微分方程

□ 显式微分方程-刚性问题

刚性问题

有一类常微分方程，其解的分量有的变化很快，有的变化很慢，且相差悬殊，这就是所谓的刚性问题(Stiff)。

对于刚性问题，数值解算法必须取很小步长才能获得满意的结果，导致计算量会大大增加。解决刚性问题需要有专门方法。非刚性算法可以求解刚性问题，只不过需要很长的计算时间。

➤ 示例2:

$$\begin{aligned}y' &= y^2 - y^3 \\ y(0) &= \lambda \\ 0 \leq t &\leq \frac{2}{\lambda}\end{aligned}$$

其中y是t的函数

```
function model2(t, y)
    dydt = y.^2 - y.^3
    return dydt
end

lamda=0.01
```

4.2.2 常微分方程的数值求解-显式微分方程

```
lamda=0.01
```

```
t, y, = ode45(model2, [0 2/lamda], lamda)
@time t, y, = ode45(model2, [0 2/lamda], lamda)
# 0.001018 seconds (9.56 k allocations: 393.984 KiB)
@show length(t)
#157
```

运行时间: 0.001018s

点数: 157

结果表明: 这时这个方程不算很刚性

```
lamda=1e-5
```

```
t, y, = ode45(model2, [0 2/lamda], lamda)
@time t, y, = ode45(model2, [0 2/lamda], lamda)
# 1.000353 seconds (7.83 M allocations: 866.981 MiB,
13.39% gc time, 0.35% compilation time)
@show length(t)
#120565
```

运行时间: 1.00353s

运行点数: 120565

这次时间明显加长, 计算的点数激增

结果表明: 此时这个方程表现为刚性

@time用于执行表达式的宏, 打印执行所需的时间、分配数及其字节总数

@show显示表达式和结果, 返回结果

4.2.2 常微分方程的数值求解-显式微分方程

```
lamda=1e-5

t, y, = ode23s(model2, [0 2/lamda], lamda)
@time t, y, = ode23s(model2, [0 2/lamda], lamda)
# 0.004370 seconds (32.92 k allocations: 1.612 MiB)

@show length(t)
#98
```

运行时间: 0.004370s

运行点数: 98

对于刚性问题，我们需要改变求解算法。我们选择以“s”结尾的函数，他们专门用于求解刚性方程。计算时间大大缩短，计算的点数大大减少。

4.2.3 常微分方程的数值求解-完全隐式微分方程

完全隐式常微分方程

完全隐式常微分方程的形式为： $f(t, y, y') = 0$ 。

用法为：`[t,y] = ode15i(odefun,tspan,y0,yp0,)`

-`odefun`: 为待求解方程

-`tspan`: 用于指定积分区间

-`y0,yp0`: 分别用于指定初值有 $y(t_0)$ 和 $y'(t_0)$, $f(t_0, y_0, yp_0) = 0$ 。

-`options`: 可选参数, 用于指定积分方法

`[y0mod,yp0mod] = decic(odefun,t0,y0,fixed_y0,yp0,fixed_yp0)`: 为 `ode15i` 计算一致的初始条件。
给定`y0`, `fixed_y0`取1, 否则为0, 给定`yp0`, 则`fixed_yp0`取1, 否则取0。

4.2.3 常微分方程的数值求解 - 完全隐式微分方程

完全隐式微分方程

示例:

$$\begin{cases} ty^2(y')^3 - y^3(y')^2 + t(t^2 + 1)y' - t^2y = 0 \\ y(1) = \sqrt{3/2} \longrightarrow y'(1) = ? \end{cases}$$

编写方程代码

```
function weissinger(t, y, yp)
    return @. t * y^2 * yp^3 - y^3 * yp^2 + t *
(t^2 + 1) * yp - t^2 * y
end
```

计算一致的初始条件: 使用 decic 加上 ODE15i 返回的一致初始条件, 在时间区间 [1 10] 上求解 ODE。

```
t0 = 1
y0 = sqrt(3 / 2)
yp0 = 0
y0, yp0 = decic(weissinger, t0, y0, 1, yp0, 0)
```

4.2.3 常微分方程的数值求解 - 完全隐式微分方程

完全隐式微分方程

绘制结果

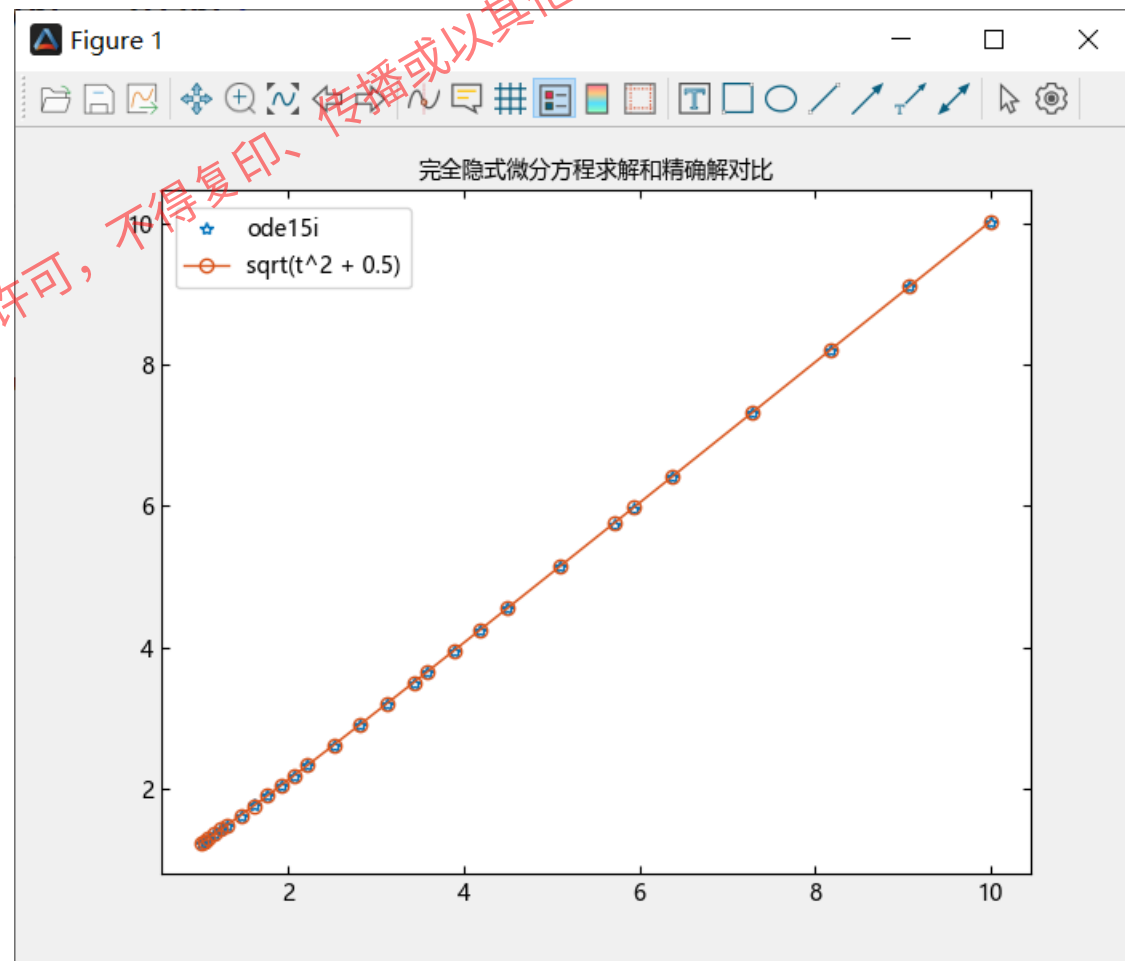
此 ODE 的精确解为

$$y(t) = \sqrt{t^2 + \frac{1}{2}}$$

绘制 ode15i 计算的数值解 y 对解析解 ytrue 的图。

#解析解

```
ytrue = @. sqrt(t^2 + 0.5)
plot(t, y, "*", t, ytrue, "-o")
legend(["ode15i", "sqrt(t^2 + 0.5)"])
```



4.2.4 常微分方程的数值求解-代数微分方程

代数微分方程-DAEs

微分方程的形式是多种多样，一般来说，很多很多高阶微分方程可以通过变量替换转化成一阶微分方程组，即可以写成下面的形式①：

$$M(t, y)y' = F(t, y)$$

$M(t, y)$ 称为质量矩阵，如果其非奇异的话，上式可以写成式子②：

$$y' = M(t, y)^{-1}F(t, y)$$

将②式右半部分用odefun表示出来，就是ode45，ode23等常微分方程初值问题求解的输入参数odefun。

如果质量矩阵奇异的话，①式称为微分代数方程组 (differential algebraic equations, DAEs.)

可以利用求解刚性微分方程的函数如ode15s、ode23s等来求解

从输入形式上看，求解DAEs和求解普通的ODE很类似，主要区别是需要给**微分方程求解器指定质量矩阵**。

隐式微分方程无法写成①或者②的形式

4.2.4 常微分方程的数值求解 - 代数微分方程

所有 ODE 求解器都可以解算 $y' = f(t, y)$ 形式的方程组，或涉及质量矩阵 $M(t, y)y' = f(t, y)$ 的问题。
ode23s 求解器只能解算质量矩阵为常量的问题。

ode15s 和 **ode23t** 可以解算具有奇异质量矩阵的问题，使用 odeset 的 Mass 选项指定质量矩阵。

ode15s的求解：

```
[t,y] = ode15s(odefun,tspan,y0,options)
```

-odefun：为待求解方程

-tspan：用于指定积分区间

-y0：分别用于指定初值。

-options：使用 odeset 函数创建的参数，定义的积分设置

Tips: options = odeset(Name,Value,...) 创建 options 结构体，您可以将其作为参数传递给 ODE 和 PDE 求解器

4.2.4 常微分方程的数值求解-代数微分方程

求解代数微分方程示例：

$$\begin{cases} y_1' = -0.2y_1 + y_2y_3 + 0.3y_1y_2 \\ y_2' = 2y_1y_2 - 5y_2y_3 - 2y_2^2 \\ y_1 + y_2 + y_3 = 1 \end{cases}$$

微分方程

代数方程

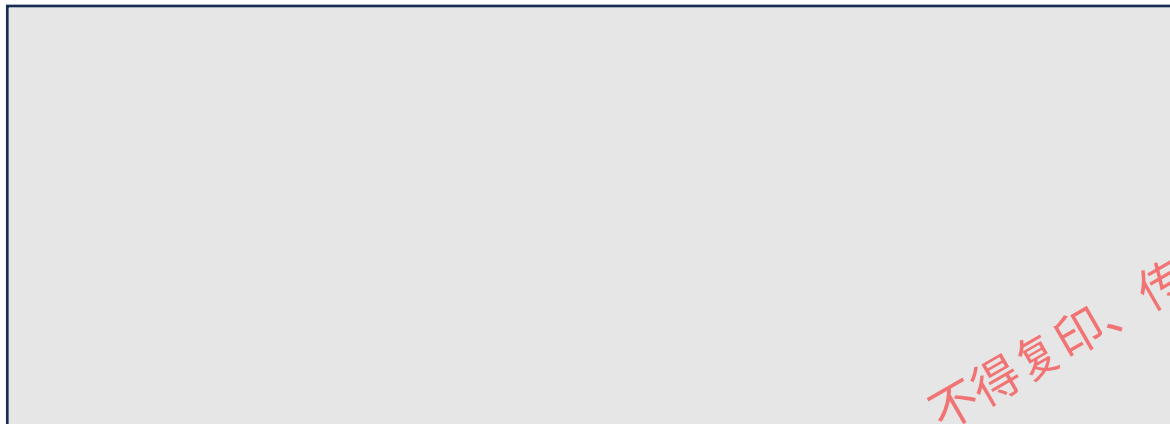
初值： $y_1(0) = 0.8, y_2(0) = 0.1, y_3(0) = 0.1$

切换为矩阵形式的表示：

$$M(t, y)y' = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} dy_1 / dt \\ dy_2 / dt \\ dy_3 / dt \end{bmatrix} = \begin{bmatrix} -0.2y_1 + y_2y_3 + 0.3y_1y_2 \\ 2y_1y_2 - 5y_2y_3 - 2y_2^2 \\ y_1 + y_2 + y_3 - 1 \end{bmatrix} = F(t, y)$$

4.2.4 常微分方程的数值求解-代数微分方程

- 定义待求解的方程



- 定义初始值向量y0

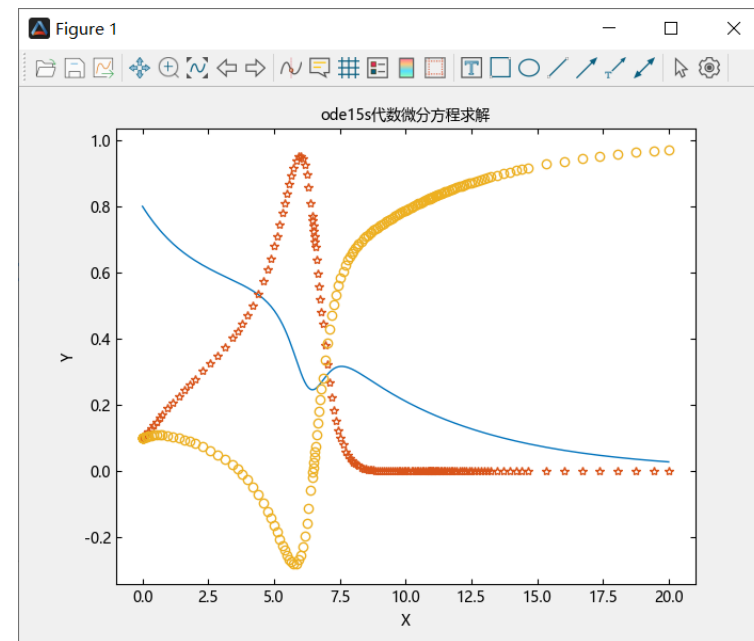
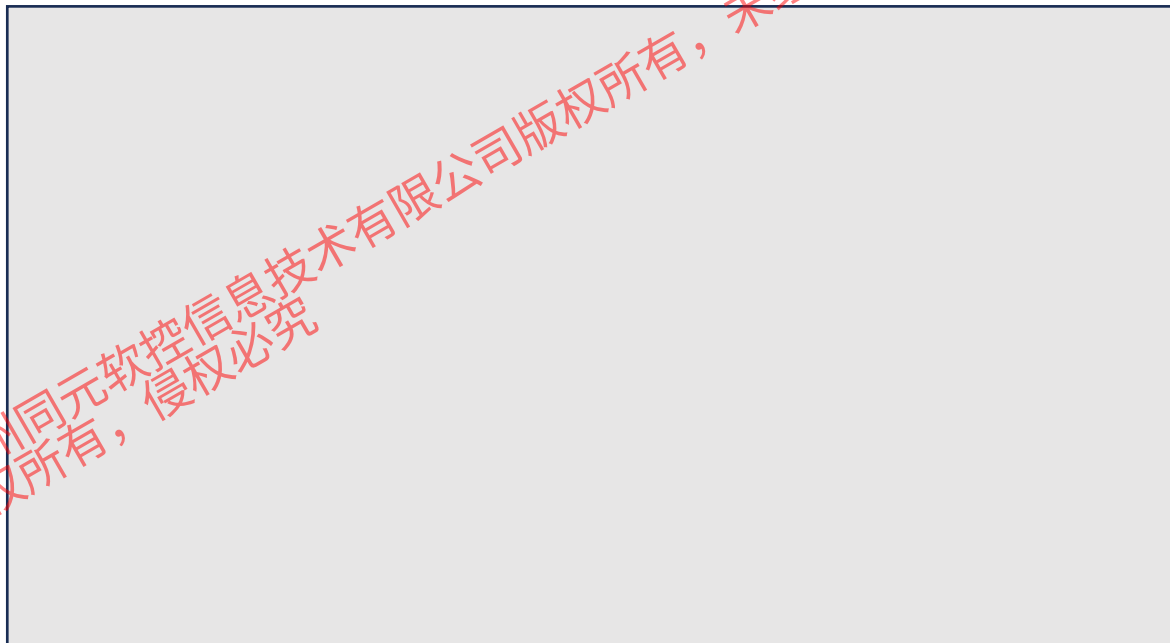
- 定义矩阵M

- 定义区间

- 定义结构体

- 调用ode15s函数

- 绘图



4.3.1 微分方程的边界值求解

微分方程分为**初值问题**和**边值问题**。

- 初值问题是已知微分方程的初始条件，即自变量为零时的函数值，一般可以用欧拉法、龙格库塔法来求解。
- 边值问题则是已知微分方程的边界条件，即自变量在边界点时的函数值。

上节我们介绍的常微分方程，主要是微分方程的初值问题。本节介绍二阶常微分方程边值问题的建模与求解。

求解器	说明
bvp4c	求解边界值问题 - 四阶方法
bvp5c	求解边界值问题 - 五阶方法
bvpinit	得出边界值问题求解器的初始估计值

BVP 求解器的选择

软件提供了求解器 bvp4c 和 bvp5c 来求解 BVP。在大多数情况下，可以互换使用这些求解器。这些求解器之间的主要区别在于 bvp4c 实现四阶公式，而 bvp5c 实现五阶公式。

除两个求解器之间的误差容限含义以外，bvp5c 函数的使用方法与 bvp4c 完全类似。如果 $S(x)$ 近似于解 $y(x)$ ，则 bvp4c 控制残差 $|S'(x) - f(x, S(x))|$ 。这种方法间接控制真误差 $|y(x) - S(x)|$ 。使用 bvp5c 直接控制真误差。

4.3.2 微分方程的边值问题

□ 边值问题

只含边界条件作为定解条件的常微分方程求解问题，称为常微分方程的边值问题（boundary value problem）。

- 若边界条件指定二点数值，称为狄利克雷边界条件（第一类边值条件）。
- 此外也有指定二个特定点上导数的边界条件，称为诺伊曼边界条件（第二类边值条件）。
- 第三类边值条件：给出未知函数在边界上的函数值和外法线的方向导数的线性组合。

BVP 求解器 bvp4c 和 bvp5c 用于处理以下形式的 ODE 方程组：

$$y' = f(x, y)$$

其中， x 是自变量， y 是因变量， $y' = \frac{dy}{dx}$

形式：

```
sol = bvp4c(odefun,bcfun,solinit)
sol = bvp4c(odefun,bcfun,solinit,options)
sol = bvp5c(odefun,bcfun,solinit)
sol = bvp5c(odefun,bcfun,solinit,options)
```

需要编写一个将方程表示为一阶方程组的函数、一个边界条件函数和一个初始估计值函数。然后 BVP 求解器使用这三个输入来求解方程

4.3.2. 微分方程的边值问题

□ 边值问题

方程函数-odefun

示例: 在`syslab`中求解一个二阶的BVP

$$y'' + y = 0$$

该方程在区间 $[0, \pi/2]$ 上进行定义, 受限于边界 $y(0) = 0$ 且 $y(\pi/2) = 2$

编写一个用于编写方程代码的函数。使用代换法, 将方程重写为一阶方程组。

$$y1 = y$$

$$y2 = y'$$

重写

$$y'1 = y2$$

$$y'2 = -y1$$

```
function bvpfcn(x, y)
    return [y[2]; -y[1]]
end
```

或者

```
function bvpfun(x, y) # equation being solved
    yp=zeros(2,1);
    yp[1] = y[2]
    yp[2] =-y[1]
    return [y[2]; -y[1]]
end
```

4.3.2. 微分方程的边值问题

□ 边值问题

边界条件-bcfun

在两点 BVP 的最简单情形中，ODE 的解在区间 $[a, b]$ 中求得，并且必须满足边界条件

$$g(y(a), y(b)) = 0.$$

要指定给定 BVP 的边界条件：

- 编写 $res = bcfun(ya, yb)$ 形式的函数，如果涉及未知参数，则使用 $res = bcfun(ya, yb, p)$ 形式。

将此函数作为第二个输入参数提供给求解器。该函数返回 res ，这是在边界点处的解的残差值。

例如，如果 $y(a) = 1$ 且 $y(b) = 2$ ，则边界条件函数为：

```
function bcfcn(ya, yb)
    return [ya[1]; yb[1] - 2]
end
```

4.3.2. 微分方程的边值问题

□ 边值问题

初始估计值-solinit

求解 BVP 过程的**重要部分**是提供所需解的估计值。

得出 BVP 问题的解的良好初始估计值可能是求解问题**最困难**的部分。

解的初始估计值

与初始值问题不同，边界值问题的解可以是：

- 无解
- 有限个解
- 无限多个解

常规指导原则

- 确保初始估计值满足边界条件，因为解也需要满足这些边界条件有限个解。
- 尝试将关于实际问题或预期解的信息尽可能多地纳入初始估计值。
- 考虑网格点的位置（解的初始估计值的 x 坐标）。
- 如果在较小区间内有一个已知的、更简单的解，则将它用作较大区间内的初始估计值

苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

侵权必究

4.3.2. 微分方程的边值问题

□ 边值问题

初始估计值-solinit

bvpinit-得出边界值问题求解器的初始估计值函数

solinit = bvpinit(x,yinit) 使用初始网格 x 和初始解估计值 yinit 来得出边界值问题解的初始估计值。

接上个示例：通过函数句柄提供初始估计值

合理地预期方程的解是振荡的，因此使用正弦和余弦函数可以很好地得出解及其固定边界点之间导数的初始估计值。

```
function guess(x) # initial guess for y and y'
    return [sin(x); cos(x)]
end
```

使用域 $[0, \pi/2]$ 中的 10 个等距网格点和初始估计值函数创建一个解结构体。

```
xmesh = LinRange(0, pi/2, 10)
solinit = bvpinit(xmesh, guess)
```

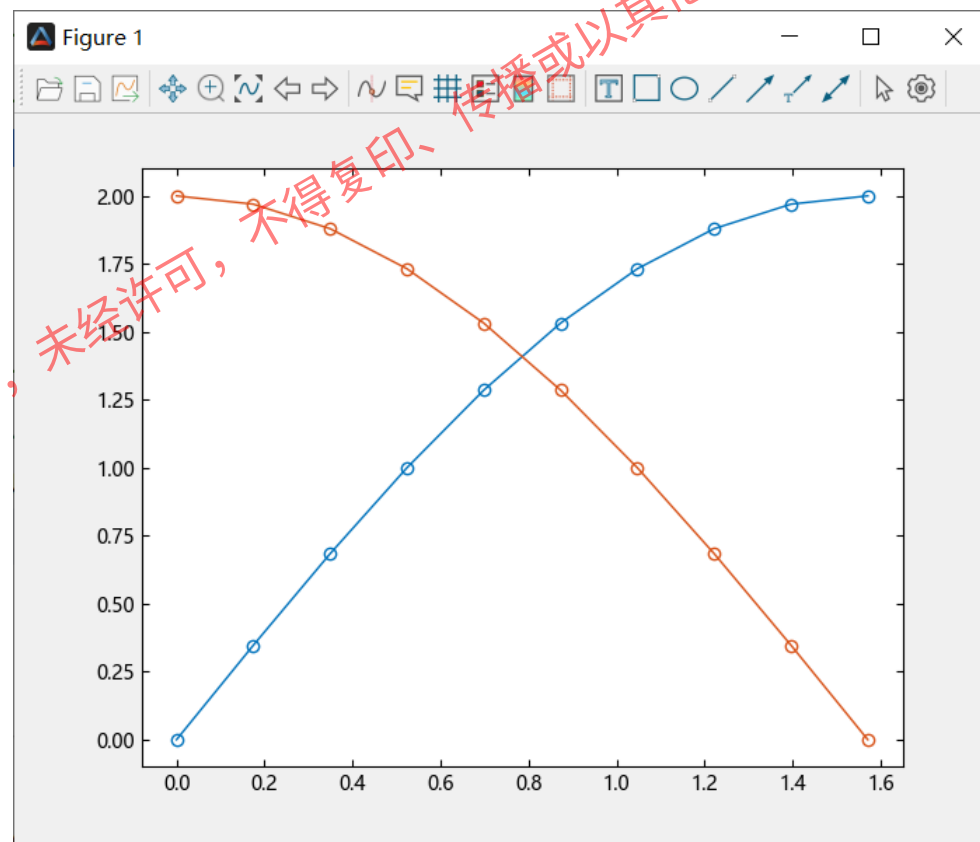
```
solinit = bvpinit(x,yinit)
solinit = bvpinit(sol,[anew bnew])
solinit = bvpinit(____,parameters)
```

4.3.2. 微分方程的边值问题

□ 边值问题

求解BVP-bvp4c

```
sol = bvp4c(bvpfun, bcfun, solinit);  
plot(sol.x, sol.y, "-o")
```



@苏州同元软控信息技术有限公司版权所有，
未经许可，不得转载、复制或传播或以其他形式使用

4.4.1 延迟（时滞）微分方程

- 在许多现实模型中，我们需要知道系统过去时刻的状态，这就形成了延迟微分方程（Delay Differential Equations）。
- 所谓延迟/时滞微分方程，指微分方程中信号不是同时发生的，除了当前时刻，还含有信号以前时刻的值。

典型（常数）	➡	$y' = f(t, y, y(t-t_1), y(t-t_2), \dots, y(t-t_n))$
变时间	➡	$y' = f(t, y, y(t-t_1), y(t-g(t)), \dots, y(t-t_n))$
中立型	➡	$y' = f(t, y, y(t-t_1), \dots, y(t-t_n), y'(t-t_{n+1}), \dots, y'(t-t_{n+m}))$

延迟（时滞）微分方程是微分方程表达式要依赖某些状态变量过去一些时刻

$$y' = f(t, y, y(t-t_1), y(t-t_2), \dots, y(t-t_n))$$

$t_1, t_2, \dots, t_n$ 是时间延迟项。既可以是常数也可以是关于 t 和 y 的函数。

当是常数时可以用 **dde23** 和 **ddesd** 来求解

4.4.1 延迟（时滞）微分方程

```
sol = dde23(ddefun,lags,history,tspan,option)
```

示例：求解延迟微分方程（常数）

$$y_1'(t) = y_1(t - 1)$$

$$y_2'(t) = y_1(t - 1) + y_2(t - 0.2)$$

$$y_3'(t) = y_2(t).$$

$$t \leq 0 \quad y_1(t) = y_2(t) = y_3(t) = 1 \quad \text{history}$$

- ddefun表示函数句柄，形式dydt=ddefun(t,y,Z):

- t 与当前t对应；y是对y(t)的近似；
- Z(:,j)是对所有延迟为 t_j 的状态变量即 $y(t - t_j)$ 的估计， $t_j = \text{lags}(j)$

- lags存储各延迟常数的向量：比如 $t_1, t_2, \dots, t_n$

- history是描述 $t \leq t_0$ 时的状态变量的值的函数，可以为句柄形式或者常数形式。

	1	0.2	
	lags1	lags2	lags3
y1	Z(1,1)	Z(1,2)	Z(1,3)
y2	Z(2,1)	Z(2,2)	Z(2,3)
y3	Z(3,1)	Z(3,2)	Z(3,3)

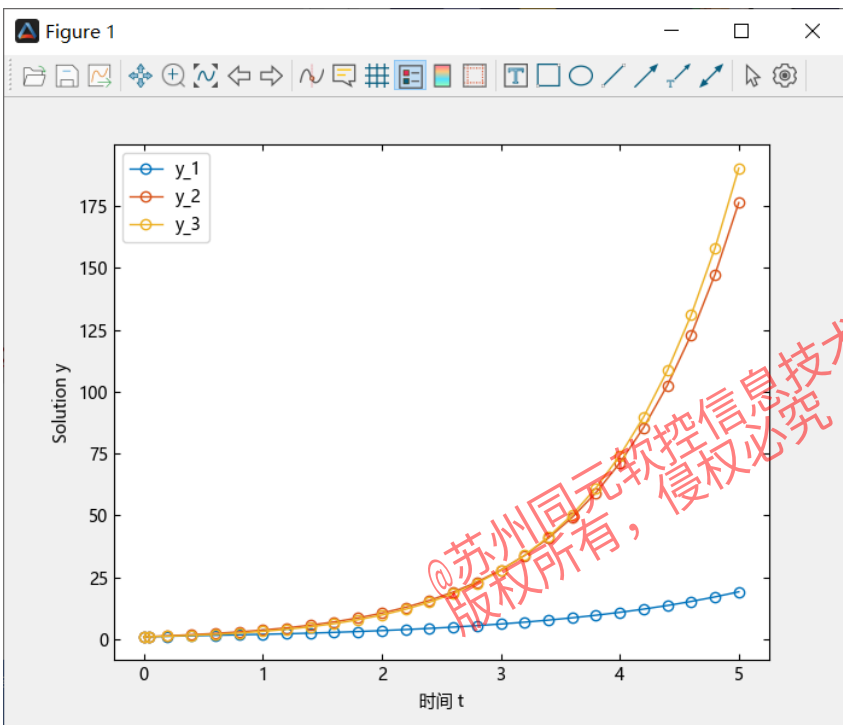
4.4.1 延迟（时滞）微分方程

$$y_1'(t) = y_1(t - 1)$$

$$y_2'(t) = y_1(t - 1) + y_2(t - 0.2)$$

$$y_3'(t) = y_2(t).$$

$$t \leq 0 \quad y_1(t) = y_2(t) = y_3(t) = 1$$



4.5 一维偏微分方程PDE

在偏微分方程 (PDE) 中，要求解的函数取决于几个变量，微分方程可以包括关于每个变量的偏导数。偏微分方程可用于对波浪、热流、流体扩散和其他空间行为随时间变化的现象建模。

pdepe 要求解的一维方程大概可分为以下两类：

带时间导数的方程是抛物型方程。例如热方程 $\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}$

不带时间导数的方程是椭圆型方程。例如拉普拉斯方程 $\frac{\partial^2 u}{\partial x^2} = 0$

求解一维 PDE

一维 PDE 包含函数 $u(x,t)$ ，该函数依赖于时间 t 和一个空间变量 x 。

PDE 求解器 pdepe 求解以下形式的一维抛物型和椭圆型 PDE 的方程组。

$$c\left(x, t, u, \frac{\partial u}{\partial x}\right) \frac{\partial u}{\partial t} = x^{-m} \frac{\partial}{\partial x} \left(x^m f\left(x, t, u, \frac{\partial u}{\partial x}\right) \right) + s\left(x, t, u, \frac{\partial u}{\partial x}\right)$$

- PDE 在 $t_0 \leq t \leq t_f$ 和 $a \leq x \leq b$ 时成立。
- 空间区间 $[a, b]$ 必须为有限值。
- m 可以是 0、1 或 2，分别对应平板、柱状或球面对称性。如果 $m > 0$ ，则 $a \geq 0$ 也必须成立。
- 系数 $f(x,t,u, \partial u/\partial x)$ 是通量项， $s(x,t,u, \partial u/\partial x)$ 是源项。
- 通量项必须取决于偏导数 $\partial u/\partial x$ 。

4.5 一维偏微分方程PDE

求解过程

要使用 pdepe 求解 PDE，您必须定义 c、f 和 s 的方程系数、初始条件、解在边界处的行为以及在其上计算解的点网格。

函数调用:

```
sol= pdepe(m,pdefun,icfun,bcfun,xmesh,tspan)
```

m 是对称常量。

pdefun 定义要求解的方程。

icfun 定义初始条件。

bcfun 定义边界条件。

xmesh 是 x 的空间值向量。

tspan 是 t 的时间值向量。

xmesh 和 tspan 向量共同构成一个二维网格，pdepe 在该网格上计算解。

©苏州同元物联信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

4.5 一维偏微分方程PDE

示例: $\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}$

初始条件: $u(x,0)=1/2$

编写方程-bdefun

编写函数 pdefun, 它计算系数 c、f 和 s 的值。

使用函数满足 $c, f, s = \text{pdefun}(x, t, u, \text{dudx})$

- pdefun 的输入参数是标量 x 和 t 以及向量 u 和 dudx。

向量 u 逼近解 u, dudx 逼近它关于 x 的偏导数。

- 输出参数 c、f 和 s 是列向量, 其元素数等于方程数。

c 存储矩阵 c 的对角线元素。

重写方程

$$1 \cdot \frac{\partial u}{\partial t} = x^0 \frac{\partial}{\partial x} \left(x^0 \frac{\partial u}{\partial x} \right)$$

可知系数值 $m = 0$ 、 $c = 1$ 、 $f = \partial u / \partial x$ 和 $s = 0$ 。此方程的函数是:

```
function heatpde(x, t, u, dudx)
    return 1, dudx, 0
end
```

4.5 一维偏微分方程PDE

示例: $\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}$

初始条件: $u(x,0)=1/2$

边界条件: $u(0,t)=0, u(1,t)=1$

初始条件-icfun

对于 $t = t_0$ 和所有 x , 解分量均满足以下形式的初始条件:

$$u(x, t_0) = u_0(x).$$

当与参数 x 一起调用时, icfun 函数会计算并返回列向量 u_0 中 x 处的解分量的初始值。 u_0 中元素的数量等于方程的数量。

$$u_0 = \text{icfun}(x)$$

重写初始方程

恒定初始条件 $u(x,0)=1/2$ 对应于以下函数:

```
function heatic(x)
    return 0.5
end
```

4.5 一维偏微分方程PDE

示例: $\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}$

初始条件: $u(x,0)=1/2$

边界条件: $u(0,t)=0, u(1,t)=1$

边界条件-bcfun

编写函数 bcfun, 它定义边界条件的项 p 和 q。使用函数满足

$$p1,q1,pr,qr = \text{bcfun}(xl,ul,xr,ur,t)$$

- ul 是左边界 $xl = a$ 处的近似解, ur 是右边界 $xr = b$ 处的近似解。
- $p1$ 和 $q1$ 是列向量, 对应于在 xl 处计算的 p 和 q 。同样, pr 和 qr 则与 xr 相对应。
- 输出 $p1$ 、 $q1$ 、 pr 和 qr 中的元素数量等于方程的数量。

当 $m > 0$ 且 $a = 0$ 时, 由于解在 $x = 0$ 附近具备有界性, 通量 f 需在 $a = 0$ 处消失。pdepe 会自动应用此边界条件, 并且忽略 $p1$ 和 $q1$ 中返回的值。

重写边界方程

以区间 $0 \leq x \leq 1$ 上的边界条件 $u(0,t)=0, u(1,t)=1$ 为例。边界条件以求解器所需形式表示为

$$u(0,t) + 0 \cdot f(0,t,u, \frac{\partial u}{\partial x}) = 0,$$

$$u(1,t) - 1 + 0 \cdot f(1,t,u, \frac{\partial u}{\partial x}) = 0.$$

在这种形式下, 很明显, 两个 q 项都为零。 p 项非零, 以反映 u 的值。对应的函数是

```
function heatbc(xl, ul, xr, ur, t)
    p1 = ul
    q1 = 0
    pr = ur .- 1
    qr = 0
    return p1, q1, pr, qr
end
```

4.5 一维偏微分方程PDE

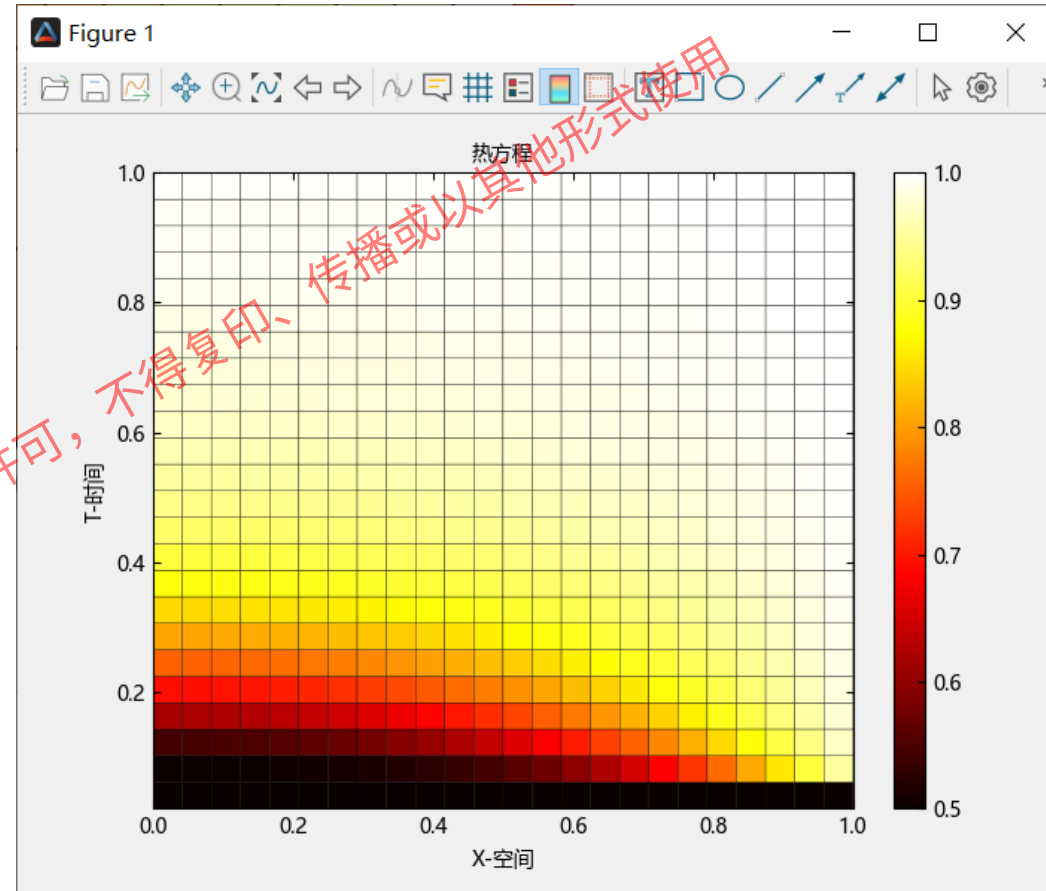
```
x = LinRange(0, 1, 25)
t = LinRange(0.02, 1, 25)
```

#求解方程

```
m=1
sol = pdepe(m, heatpde, heatic, heatbc, x, t);
u = sol[:, :, 1];
```

#可视化矩阵

```
a = pcolor(x, t, u)
colormap(a, "hot")
colorbar(gca(), a)
```



@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究



Thanks.

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究